**SVEUČILIŠTE U SPLITU**

**SVEUČILIŠNI ODJEL ZA STRUČNE STUDIJE**

Stručni prijediplomski studij Računarstvo

**PETAR TOPIĆ**

# Z A V R Š N I  R A D

# WEB TRGOVINA ZA PRODAJU INFORMATIČKE OPREME

Split, rujan 2024.

**SVEUČILIŠTE U SPLITU**
**SVEUČILIŠNI ODJEL ZA STRUČNE STUDIJE**

Stručni prijediplomski studij Računarstvo

**Predmet:** Građa i održavanje osobnih računala

# Z A V R Š N I  R A D

**Kandidat:** Petar Topić

**Naslov rada:** Web trgovina za prodaju informatičke opreme

**Mentor:** Nikola Grgić, viši predavač

Split, rujan 2024.

# Sadržaj

**Sažetak** ..... **1**

**1. Uvod** ..... **2**

**2. Korištene tehnologije** ..... **3**

2.1. HyperText Markup Language (HTML) ..... 3

2.2. Cascading Style Sheets (CSS) ..... 3

2.3. Programski jezik Javascript ..... 4

2.4. Razvojno okruženje React ..... 5

2.4.1. Model-View-Controller (MVC) arhitektura ..... 6

2.5. NoSQL baza podataka MongoDB ..... 7

2.6. Node.js ..... 9

2.7. Express.js ..... 10

**3. Izgled i funkcionalnosti aplikacije** ..... **11**

3.1. Struktura baze podataka ..... 11

3.2. Izgled korisničkog sučelja i funkcionalnosti aplikacije ..... 12

3.2.1. Aplikacija namijenjena za kupce ..... 16

3.2.2. Aplikacija namijenjena za administratore sustava ..... 37

**4. Zaključak** ..... **45**

**Literatura** ..... **46**

**Dodaci** ..... **48**

# Sažetak

Cilj ovog završnog rada jest razviti mrežnu aplikaciju za prodaju proizvoda informatičke opreme. Aplikacija korisniku omogućava pregled svih proizvoda u trgovini, pregled proizvoda određene kategorije, dodavanje tih proizvoda u listu želja te dodavanje proizvoda u košaricu, kao i kupovinu proizvoda iz košarice. Aplikacija sadrži konfigurator, čija je svrha slaganje konfiguracije računala od odabranih komponenti. Konfigurator olakšava proces slaganja konfiguracije računala tako što automatski nudi izbor komponenti koje međusobno funkcioniraju. Na primjer, kad korisnik želi odabrati matičnu ploču nakon što je odabrao procesor, konfigurator mu nudi one matične ploče koje mogu podržati odabrani procesor.



**Ključne riječi:** informatička oprema, kupovina, proizvodi, trgovina, web aplikacija



# Summary

## Web store for selling IT equipment

The purpose of this final project is to develop a web application for selling IT equipment. Application allows the user to view all products in a store, view products in certain category, adding products to wishlist, adding products to cart and buying products from cart. The application contains a configurator which purpose is to assemble a configuration of the computer from selected components. The configurator eases the proccess of assembly by automaticly offering a choice of components which work together. For example, when the user wants to choose a motherboard after choosing a processor, the configurator offers him motherboards that can support the selected processor.



**Keywords:** buying, IT equipment, products, store, web application

# 1. Uvod

Aplikacija izrađena za ovaj završni rad spada u kategoriju *e-commerce* (engl. *electronic commerce*) mrežnih aplikacija (engl. *web application*). *E-commerce* je aktivnost kupnje i prodaje proizvoda ili usluga putem interneta. Prednost *e-commerce* trgovina je ta što se kupac može nalaziti bilo gdje na svijetu sve dok ima pristup internetu. Istraživanja pokazuju da popularnost *e-commercea* kontinuirano raste te će se taj trend rasta održati u budućnosti zato što internet i njegove usluge postaju sve dostupnije.

Iz perspektive kupaca, kupnja putem interneta nudi znatno veći izbor proizvoda u usporedbi s fizičkim trgovinama, što povećava vjerojatnost da će kupci pronaći proizvode koji im odgovaraju. Uzevši za primjer informatičku opremu, online trgovine omogućuju pristup širokom asortimanu proizvoda koji možda nisu dostupni u lokalnim trgovinama.

Također, pandemija bolesti COVID-19 značajno je utjecala na porast popularnosti online kupovine. Ponajviše jer su ljudi strahovali od zaraze pa su znatno manje odlazili u fizičke trgovine, a to ih je, na koncu, i potaknulo na istraživanje i korištenje online trgovina.

U drugom poglavlju nalazi se opis tehnologija korištenih za izradu aplikacije te njihove karakteristike. Tehnologije korištene za izradu korisničkog sučelja (engl. *frontend*) su: HTML, CSS, JavaScript i React. Za potrebe baze podataka korištena je NoSQL baza podataka naziva MongoDB. Razvojna okruženja (engl. *framework*), korištena za razvoj pozadinskih funkcionalnosti (engl. *backend*), Node.js i Express.js. Potom, treće poglavlje opisuje konačno korisničko sučelje (engl. *user interface*) i sve funkcionalnosti aplikacije. I na koncu, zaključak donosi zaokruženi osvrt na cjelokupan rad uz potencijalne nadogradnje za aplikaciju.

# 2. Korištene tehnologije

## 2.1. HyperText Markup Language (HTML)

HyperText Markup Language (HTML) prezentacijski je jezik koji se koristi za izradu mrežnih stranica (engl. *web site*). Postao je popularan zbog jednostavne sintakse i lakog učenja. Pomoću njega stvaraju se hipertekstualni dokumenti koji su osnova svake mrežne aplikacije (engl. *web application*). Glavna zadaća HTML jezika jest uputiti internetski preglednik (engl. *web browser*) kako da prikaže hipertekstualne dokumente. Također, HTML se ne može smatrati programskim jezikom jer se pomoću njega ne može izvršiti nikakav zadatak, kao na primjer zbrajanje dvaju cijelih brojeva. Ipak, može se koristiti za izradu funkcionalnih mrežnih aplikacija. Međutim, takva mrežna aplikacija sastojat će se samo od statičkih elemenata kao što su tekst, slike i videa. Da bi se na mrežnoj stranici promijenila boja pozadine, dodali fontovi, dodale razne animacije i slične opcije, uz HTML, može se koristiti Cascading Style Sheets (CSS). CSS će detaljnije biti opisan u idućem poglavlju [1].

## 2.2. Cascading Style Sheets (CSS)

CSS stilski je jezik koji se koristi za oblikovanje HTML datoteka te za uređivanje izgleda i rasporeda mrežnih stranica [2].

CSS kôd može se pisati na više načina:

- u zaglavlju HTML datoteka – CSS kôd piše se između oznaka `<style>` i `</style>` unutar HTML zaglavlja `<head>`.

- unutar samih HTML oznaka – CSS kôd piše se izravno u HTML oznaku te se primjenjuje samo na taj element.

- u odvojenoj datoteci – CSS kôd može se pisati u odvojenu datoteku s nastavkom `.css`. Da bi datoteku uključili u HTML datoteku, potrebno je definirati lokaciju CSS datoteke u zaglavlju HTML datoteke. Ovaj je način najčešći i najefikasniji zato što je CSS kôd razdvojen od HTML kôda, što rezultira boljoj čitljivosti kôda [3].

## 2.3. Programski jezik Javascript

Programski jezik JavaScript koristi se za dodavanje dodatnih funkcionalnosti mrežnim stranicama. JavaScript je iznimno popularan programski jezik te se trenutno nalazi na vrhu ljestvice najpopularnijih programskih jezika [4]. Svoju popularnost stekao je zahvaljujući jednostavnoj sintaksi koja početnicima omogućuje olakšano razvijanje vlastitih interaktivnih mrežnih aplikacija (engl. *interactive web application*). Također, još jedan faktor koji uveliko utječe na njegovu popularnost, jest širok spektar područja u kojem se može upotrebljavati. Osim što se koristi za razvoj mrežnih aplikacija, može se koristiti i za razvoj mobilnih aplikacija, desktop aplikacija, *backend* aplikacija i *embedded* aplikacija [5]. Neke od osnovnih funkcija JavaScript programskog jezika su: spremanje vrijednosti u varijable, operacije s brojevima, operacije sa stringovima, pokretanje kôda kao odgovor na određene događaje (npr. mijenjanje sadržaja kada korisnik klikne na botun) i slično.

JavaScript interpreterski je jezik više razine. Većina modernih JavaScript interpretera koristi tehniku zvanu `Just-in-time compiling` koja značajno ubrzava izvršavanje programa. Proces kompajliranja JavaScript kôda odvija se tako da se izvorni kôd (engl. *source code*) kompajlira u strojni kôd (engl. *machine code*) prije izvođenja programa te se tijekom izvođenja programa kôd interpretira. [7].

Jedna od bitnijih stvari na koju treba paziti tijekom pisanja JavaScript kôda je ta da JavaScript ne podržava tipove podataka, što znači da svaka varijabla može biti bilo kojeg tipa [6].

JavaScript kôd uključuje se u mrežnu stranicu na sličan način kao i CSS kôd. Kao što kôd CSS koristimo element `<link>` za učitavanje CSS datoteke, tako za JavaScript koristimo element `<script>`.

Javascript kôd može se pisati na više načina:

- u zaglavlju HTML datoteke – JavaScript kôd piše se između oznaka `<script>` i `</script >` unutar HTML zaglavlja `<head>`.

- na kraju tijela HTML datoteke – JavaScript kôd piše se između oznaka `<script>` i `</script>` na kraju HTML tijela `<body>`.

- unutar samih HTML oznaka – JavaScript kôd može se nalaziti u HTML elementima te se odnosi samo na taj element. Primjer gdje se JavaScript kôd piše na ovaj način jest kad se HTML botunu želi pripojiti funkcija koja će se aktivirati klikom na botun. `<button onclick="createParagraph()">Click me!</button>`.

- u odvojenoj datoteci – JavaScript kôd može se pisati u odvojenu datoteku s nastavkom *.js*. Da bi se datoteka uključila u HTML dokument, potrebno je definirati njezinu lokaciju u zaglavlju HTML dokumenta ili na kraju tijela HTML dokumenta. Ovaj je način najčešći i najefikasniji zato što je JavaScript kôd razdvojen od HTML kôda, što rezultira boljoj čitljivosti kôda. Razlika između ova dva pristupa učitavanja JavaScript datoteka jest ta što kad se JavaScript uključi u zaglavlju HTML datoteke, tada se JavaScript učitava prije generiranja sadržaja mrežne stranice. S druge strane, kad se JavaScript uključi u tijelu HTML datoteke, tada se JavaScript učitava nakon generiranja sadržaja mrežne stranice [6].

## 2.4. Razvojno okruženje React

React (također poznat pod nazivom React.js ili ReactJS) je JavaScript biblioteka (engl. *library*) za izradu korisničkih sučelja baziranih na komponentama [8]. React je besplatna biblioteka otvorenog kôda (engl. *open source*) koju je razvio Facebook 2013. godine. Koristi se za izradu aplikacija na jednoj stranici (engl. *single page applications*), za razvoj mobilnih aplikacija pomoću React Native biblioteke te za razvoj aplikacija generiranih na poslužitelju (engl. *server rendered applications*) pomoću razvojnog okruženja Next.js.

Prilikom razvoja mrežnih aplikacija s većim brojem stranica poželjno je koristiti dodatne biblioteke.

Komponente u React okruženju označuju odlomke kôda koji su modularni, koriste se na različitim mjestima u aplikaciji. Glavna prednost korištenja komponenti je ta što se njihovim korištenjem aplikacije razbijaju na manje dijelove te su samim tim aplikacije čitljivije i lakše za održavanje. Komponente se često pišu tako da jedna roditeljska (engl. *parent*) komponenta sadrži više dječjih (engl. *child*) komponenti. Ovaj pristup pomaže u procesu otklanjanja greški (engl. *debugging*) jer *developer* može lakše otkriti u kojem se komadiću kôda dogodila greška. Komponente se u većini slučajeva pišu koristeći JSX sintakse koja je kombinacija JavaScript programskog jezika i HTML-a. Da bi se komponenti

dodale određene vrijednosti, koriste se takozvana svojstva (engl. *props*). Korištenjem svojstava vrijednosti se mogu prenositi između komponenti. Kod prenošenja vrijednosti između dviju ili više komponenti prijenos se odvija samo u jednom smjeru. To znači da roditeljska komponenta ne može dobiti vrijednosti od dječje komponente [8] [9].

React komponente mogu se deklarirati kao funkcijske komponente ili komponente klase. Funkcijske se komponente deklariraju pomoću funkcija ili *arrow* funkcija, a komponente klase deklariraju se pomoću klasa [8] [9].

Jedna od glavnih prednosti Reacta je ta što ima ugrađen virtualni DOM koji uvelike uvećava performanse mrežnih stranica naspram onih napravljenih u JavaScript programskom jeziku. Glavna razlika između DOM-a i virtual DOM-a je ta što virtualni DOM uspoređuje novo stanje mrežne stranice sa starim stanjem te ažurira samo one dijelove mrežne stranice koji su se promijenili dok DOM ažurira cijelu mrežnu stranicu. Taj postupak uvelike uvećava performanse mrežnih stranica jer se ne ažurira sadržaj koji se nije promijenio [9].

### 2.4.1. Model-View-Controller (MVC) arhitektura

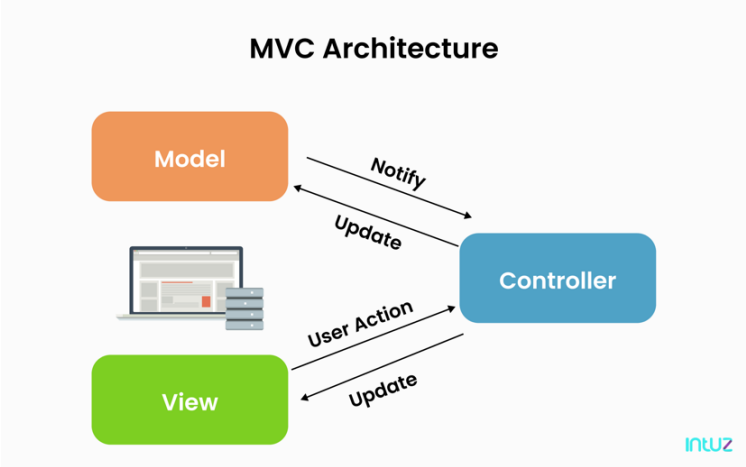
Model-View-Controller (MVC) arhitektura je način dizajniranja aplikacija koji razdvaja aplikacije na tri dijela. Svaki od tih dijelova ima određenu svrhu u aplikaciji. Često se koristi kod izrade mrežnih aplikacija. Cilj MVC arhitekture je razdvojiti komponente mrežnih aplikacije zadužene za bazu podataka, prezentacije podataka i obradu zahtjeva korisnika:

- *Model* – Koristi se kao sučelje prema podacima koji se nalaze u bazi podataka.

- *View* – Koristi se za prikaz podataka iz modela na korisničkom sučelju. Prima podatke od *Modela*, oblikuje ih te na kraju prikazuje na korisničkom sučelju.

- *Controller* – Nalazi se između *Modela* i *Viewa*. zadužen je za logiku koja se odvija u pozadini aplikacije kada korisnik izvrši određeni zahtjev (engl. *request*). U trenutku zaprimanja zahtjeva od korisnika *Controller* će primljeni zahtjev prvo obraditi izvršavanjem promjena nad Modelom, a potom generirati odgovor (engl. *response*) koji će proslijediti *Viewu* te će ga *View* prezentirati korisniku [10].

U Model View Controller (MVC) arhitekturi, React se svrstava u *View* dio koji je zaslužan za prikaz podataka iz modela na korisničkom sučelju. Slika 1 prikazuje princip rada MVC arhitekture.




**Slika 1**: Princip rada MVC arhitekture [11]

## 2.5. NoSQL baza podataka MongoDB

MongoDB baza je podataka koja se klasificira kao NoSQL baza podataka jer ne koristi SQL upite niti ima tablice kao SQL baze podataka, već funkcionira na principu dokumenata. Svi dokumenti u bazi spremljeni su u određene kolekcije koje se koriste umjesto tablica. U kolekcijama se nalaze dokumenti koji predstavljaju elemente kolekcije, to jest redove tablice. Dokumenti su struktura koja podsjeća na JSON strukturu te oni sadrže podatke koji su određeni strukturom dokumenta. Dokumenti su strukturirani slično kao JSON dokumenti, to jest kao ključ-vrijednost parovi (engl. *key value pair*) [12].

```
{
  "_id": {
    "$oid": "664c846a304bd34ed1a3abc8"
  },
  "userId": "664c83af304bd34ed1a3ab9d",
  "products": [],
  "createdAt": {
    "$date": "2024-05-21T11:24:26.489Z"
  },
  "updatedAt": {
    "$date": "2024-06-11T18:17:19.237Z"
  },
  "__v": 0
}
```

**Ispis 1**: Primjer tablice u bazi podataka MongoDB

Na ispisu 1 nalazi se primjer dokumenta koji se nalazi u kolekciji `wishlist` u bazi podataka. Dokument sadrži sljedeće informacije:

1. Polje `_id` – vrijednost se automatski generira pri dodavanju dokumenta u kolekciju.

2. Polje `userId` – u njemu se sprema primarni ključ korisnika na koji se odnosi određeni dokument. Tip podataka koje to polje prihvaća je string te nema ograničenje najveće količine znakova koji se mogu unijeti.

3. Polje `products` – u njemu se spremaju svi dokumenti iz kolekcije `products` koji predstavljaju proizvode koji se nalaze na korisnikovoj listi želja. Sadrži potpune kopije proizvoda, a ne samo primarni ključ. Polje `products` je tipa niz (engl. *array*), što omogućuje relaciju jedan prema više (engl. *one to many relation*) tako što se u njemu može nalaziti više proizvoda koji se odnose na jednu listu želja (engl. *wishlist*) za jednog korisnika.

4. Polje `createdAt` – automatski se generira tijekom izrade dokumenta u kolekciji.

5. Polje `updatedAt` – automatski se generira tijekom svakog uređivanja dokumenta u kolekciji.

6. Polje `__v` – označuje verziju dokumenta. Polje se automatski postavlja na vrijednost 0 te se inkrementira za 1. U ovoj se aplikaciji to polje ne koristi pa je zato na postavljenoj verziji.

Ako se želi dohvatiti određeni element iz tablice, upit (engl. *query*) se piše kao što se piše JSON objekt.

```
{name: „Procesori"}
```

**Ispis 2:** Primjer upita za MongoDB bazu podataka

Na ispisu 2 prikazan je primjer upita koji dohvaća kategorije koje imaju naziv `Procesori`.

MongoDB s vremenom je postao sve popularniji zbog svoje lakoće korištenja jer ne koristi SQL sintaksu koja za početnike može biti komplicirana. Također, neki od važnijih faktora koji su utjecali na rast popularnosti MongoDB baze podataka su cijena i mogućnost skaliranja baze. MongoDB baze podataka spremaju se u poslužitelje koji se nazivaju klasteri (engl. *cluster*) te je njihova cijena poprilično niža od klasičnih SQL baza podataka [13].

Relacije u MongoDB bazi podataka rade se na drugačiji način nego kod klasičnih SQL baza podataka. Kod MongoDB baze podataka, praksa je relacije stvarati spremanjem čitavog dokumenta koji se želi referencirati, umjesto primarnog ključa dokumenta. U nekim se slučajevima mogu spremati primarni ključevi. Taj proces zove se referenciranje. Međutim, takav pristup zahtijeva naknadne upite za dohvat referenciranih elemenata [14].

## 2.6. Node.js

Node.js razvojno je okruženje otvorenog koda za programski jezik JavaScript koje omogućava pokretanje JavaScript programa izvan internetskih preglednika. Pokreće ga V8 JavaScript motor (engl. *engine*) razvijen od strane Googlea [15].

Node.js koristi se za izradu mrežnih poslužitelja koristeći JavaScript i ugrađene module koji mu pružaju razne funkcionalnosti. Moduli omogućuju: pristup datotečnom sustavu, korištenje mrežnih protokola, korištenje kriptografskih funkcija te mnoge druge funkcije.

Node.js programerima pruža skup asinkronih I/O funkcija u svojoj standardnoj biblioteci koje sprječavaju blokiranje izvođenja programa. Kad Node.js izvodi neku I/O operaciju poput pristupa bazi podataka (engl. *database*) ili datotečnom sustavu (engl. *file system*) umjesto blokiranja niti Node.js će nastaviti s izvođenjem te će se vratiti na asinkronu funkciju kad odgovor bude spreman [16].

Node.js radi u petlji događaja s jednom niti (engl. *single thread*), što mu omogućava da podrži desetke tisuća istovremenih spajanja (engl. *connection*) bez potrebe za prespajanjem konteksta (engl. *context switching*) [17].

Prespajanje konteksta proces je spremanja stanja procesora ili niti da bi se naknadno to spremljeno stanje moglo obnoviti kako bi se nastavilo izvođenje s mjesta gdje je ono zaustavljeno. Prespajanjem konteksta se troši vrijeme i usporava rad programa [18].

## 2.7. Express.js

Express.js besplatno je razvojno okruženje otvorenog koda za Node.js. Koristi se za razvoj mrežnih aplikacija, REST API-a i aplikacija u stvarnom vremenu (engl. *real time*). Express.js nudi veliki broj značajki koje povećavaju produktivnost i pojednostavljuju razvoj mrežnih aplikacija. Najznačajnije su značajke posredne funkcije (engl. *middleware*) i sustav usmjeravanja (engl. *routing*) koje developerima omogućuju definiranje ruta (engl. *route*) za obrađivanje dolaznih zahtjeva HTTP (GET, POST, PUT, DELETE) te definiranje posrednih funkcija za presretanje zahtjeva [19].

# 3. Izgled i funkcionalnosti aplikacije

## 3.1. Struktura baze podataka

Tijekom razvoja aplikacija, prva faza je dizajniranje i izrada modela baze podataka. Kod dizajniranja baze podataka potrebno je odlučiti od kojih tablica će se baza podataka sastojati te koji će se podaci spremati u tablice. Također treba odrediti međusobne odnose između tablica imajući na umu sve zahtjeve aplikacije. Loše postavljeni odnosi između tablica mogu u daljnjem procesu razvoja stvoriti razne probleme prilikom dodavanja podataka u tablice te bih se tada struktura baze podataka morala popravljati ili čak iznova raditi. Također, kako aplikacija s vremenom raste, bit će teže raditi promjene na modelu baze podataka. Zbog toga procesu izrade modela baze podataka treba posvetiti dovoljno pažnje i vremena da bih se ostale faze razvoja aplikacije mogle odvijati nesmetano.

U nastavku su navedene smjernice na koje je potrebno obratiti pažnju tijekom dizajniranja baze podataka:

- Potrebno je riješiti sve oblike redudancije bez narušavanja integriteta podataka to jest rastaviti bazu na više manjih tablica i postaviti odnose između tablica.

- Potrebno je odabrati odgovarajuće tipove za podatke koji će biti pohranjeni u tablici.

- Kod rukovanja osjetljivim podacima potrebno je poduzeti mjere sigurnosti poput enkripcije podataka kako osjetljivi podaci ne bi bili vidljivi i dostupni. To se u većini slučajeva odnosi na lozinke korisnika, ali se može odnositi i na druge podatke.

Na dodatku 1 nalazi se konačni izgled modela baze podataka dizajniranog za potrebe izrade aplikacije. Navedene smjernice implementirane su na sljedeći način:

- Redundantni podaci koji bi se inače duplicirali podijeljeni su u manje tablice te su povezani relacijskim vezama.

- Odabir tipa podataka za podatke pohranjene u tablicama. Za tipove podataka u tablicama nisu korišteni nikakvi dodatni tipovi podataka osim onih koji su ugrađeni u MongoDB bazu podataka. S obzirom na to da MongoDB ne podržava

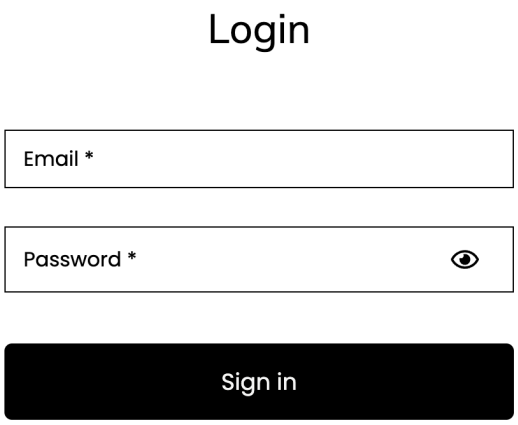
validaciju podataka validacija je odrađena na *frontend* i *backend* dijelu aplikacije. Neki od tipova podataka koji su korišteni su string, double, Date, boolean, object i Array.

- Nakon procesa registracije korisnika njegova se lozinka enkriptira koristeći metodu koja korisnikovu unesenu lozinku pretvara u *hash* kôd kako bi se lozinka sigurno pohranila u bazi podataka.

## 3.2. Izgled korisničkog sučelja i funkcionalnosti aplikacije

Aplikacija je podijeljena na dva dijela, to jest dvije manje aplikacije. Prva aplikacije namijenjena je za kupce, a druga za administratore sustava. Obje aplikacije se pokreću zasebno.

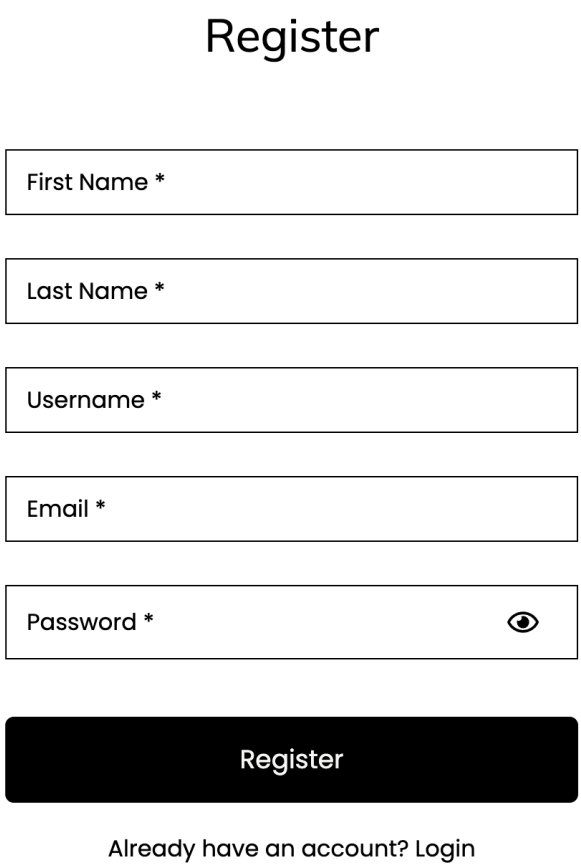
Uvjet za korištenje aplikacije namijenjene za administratore jest uspješna registracija korisničkog računa i prijava korisnika. Podatak koji odlučuje o tome može li korisnik pristupiti aplikaciji za administratore se zove `isAdmin`. Podatak je tipa boolean te prima vrijednosti `true` ili `false`. Podatak je inicijalno postavljen na `false` te ga administrator može mijenjati ovisno o tome želi li korisniku dopustiti pristup aplikaciji za administratore sustava. Kad je podatak postavljen na `true` korisniku se dozvoljava pristup u aplikaciju za administratore sustava. Na slici 2 nalazi se forma za prijavu korisnika u aplikaciju namijenjenu za administratore sustava.



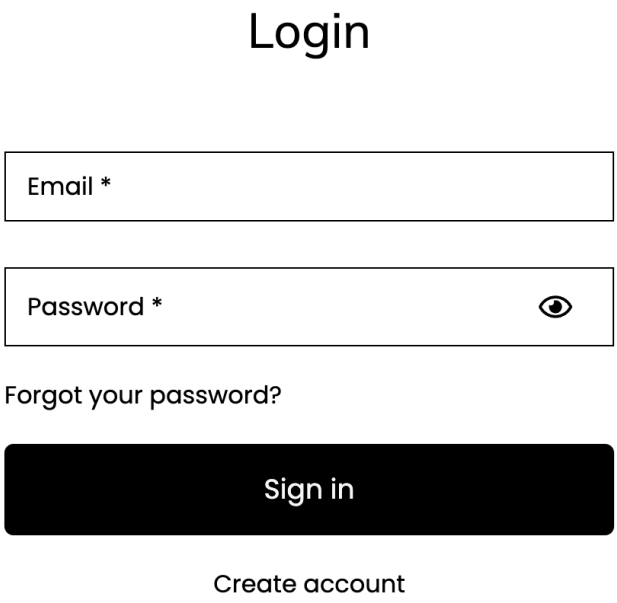

**Slika 2:** Forma za prijavu korisnika u aplikaciju namijenjenu za administratore sustava

Nakon uspješne autentikacije, korisnika se preusmjerava na početnu stranicu aplikacije za administratore sustava. Početna stranica će biti objašnjena u jednom od nadolazećih poglavlja.

Za korištenje aplikacije namijenjene za korisnike ne postoji uvjet te joj svaki korisnik može pristupiti. Korisnik ne mora biti prijavljen u sustav da joj pristupi. Jedino što prijava u sustav omogućava korisniku je bržu kupovinu proizvoda te dodavanje proizvoda u listu želja (engl. *wishlist*). Brži proces kupovine proizvoda mu je omogućen tako što korisnik može unijeti svoje podatke kao na primjer ime, prezime, adresu za dostavu narudžbe i slično. Kad korisnik unese svoje podatke oni će se automatski popuniti tijekom koraka kupovine gdje korisnik unosi podatke za dostavu proizvoda i računa čime korisnik štedi na vremenu jer ne mora ponovno unositi podatke. Registrirani korisnik također može dodati proizvode koji mu se sviđaju u listu želja te ih poslije lako dodati u narudžbu bez potrebe da ih ponovno traži u aplikaciji. Na slikama 3 i 4 nalaze se forme za prijavu i registraciju korisnika.

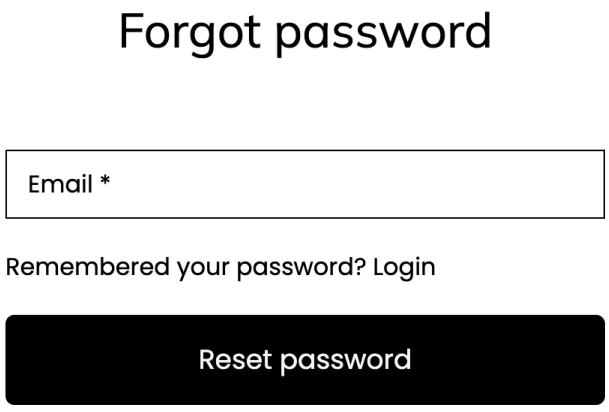



**Slika 3:** Forma za registraciju korisnika



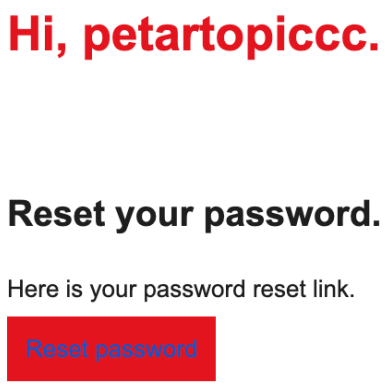

**Slika 4:** Forma za prijavu korisnika u aplikaciju namijenjenu za korisnike

Za razliku od forme za prijavu korisnika u aplikaciju za administratore forma za prijavu korisnika u aplikaciju namijenjenu za korisnike ima dodatnu stavku `Forgot your password?` (hrv. *Zaboravili ste lozinku?*) koja korisniku omogućava ponovno postavljanje lozinke u slučaju da je korisnik zaboravio svoju lozinku. Na slici 5 prikazan je izgled forme za ponovno postavljanje lozinke.



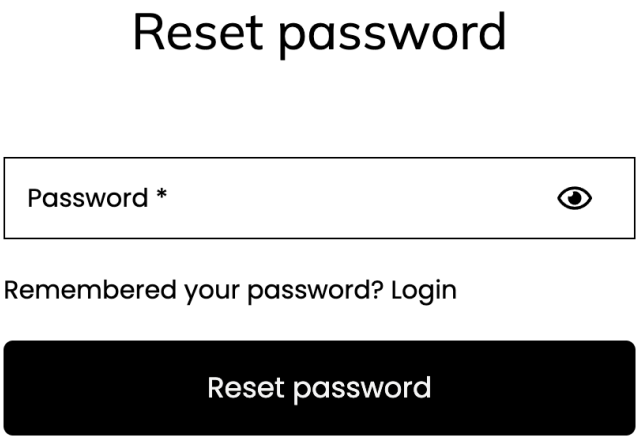

**Slika 5:** Forma za ponovno postavljanje lozinke

Nakon što korisnik unese adresu elektronske pošte koju je koristio tijekom registracije svog računa te klikne na botun `Reset password`, na e-mail će mu doći poruka za ponovno postavljanje lozinke koja je prikazana na slici 6.




**Slika 6:** Poruka za ponovno postavljanje lozinke

Klikom na botun `Reset password` otvara se stranica u novom prozoru na kojoj korisnik može ponovno postaviti svoju lozinku te se nakon toga može prijaviti u aplikaciju s novom lozinkom. Izgled stranice za postavljanje nove lozinke prikazan je na slici 7.




**Slika 7:** Forma za postavljanje nove lozinke

Nakon uspješne autentikacije korisnika se preusmjerava na početnu stranicu aplikacije koja je namijenjena za kupce. Početna stranica će biti objašnjena u idućem poglavlju.

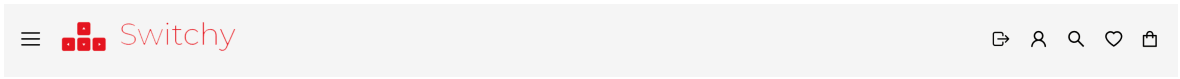
### 3.2.1. Aplikacija namijenjena za kupce

Na slici 8 prikazan je izgled početne stranice na kojoj se nalazi navigacijska traka te dijaprojekcija s promoviranim proizvodima.




**Slika 8:** Početna stranica aplikacije

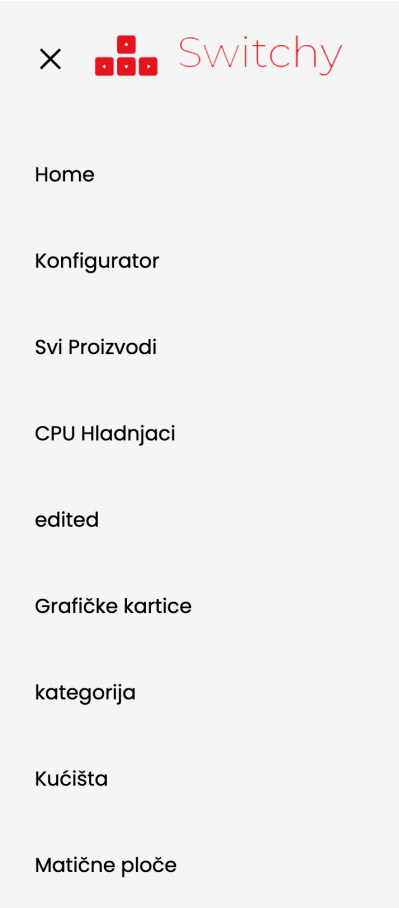
U slučaju da je korisnik prijavljen u sustav prikazuje mu se dodatna opcija za odjavu iz sustava prikazana na slici 9.




**Slika 9:** Izgled navigacijske trake za prijavljene korisnike

U navigacijskoj traci se nalaze sljedeće opcije:

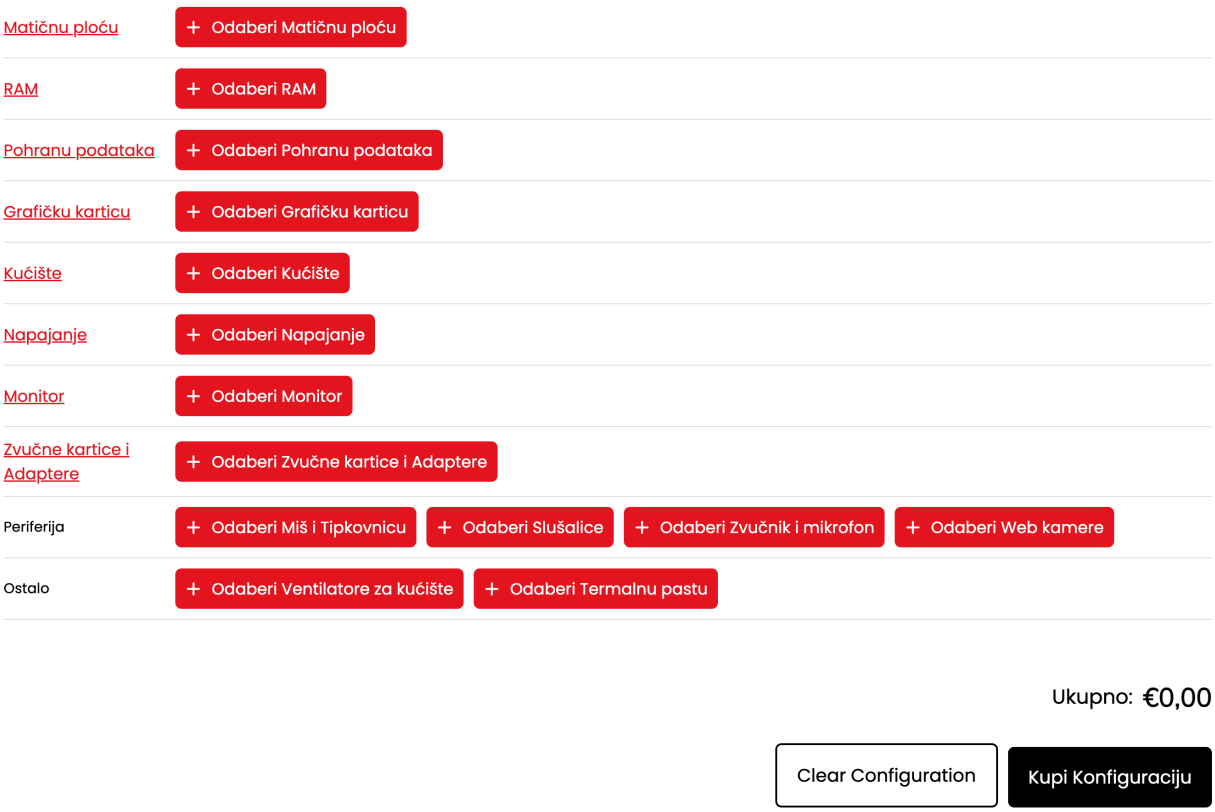
- `Hamburger menu` – Klikom na ikonicu otvara se izbornik u kojem se nalaze poveznice za početnu stranicu, konfigurator, svi proizvodi koji se nalaze u aplikaciji i pojedinačne kategorije proizvoda. Klikom na određenu poveznicu korisnika se preusmjerava na stranicu koju je odabrao. Izgled izbornika prikazan je na slici 10.




**Slika 10:** Izgled izbornika

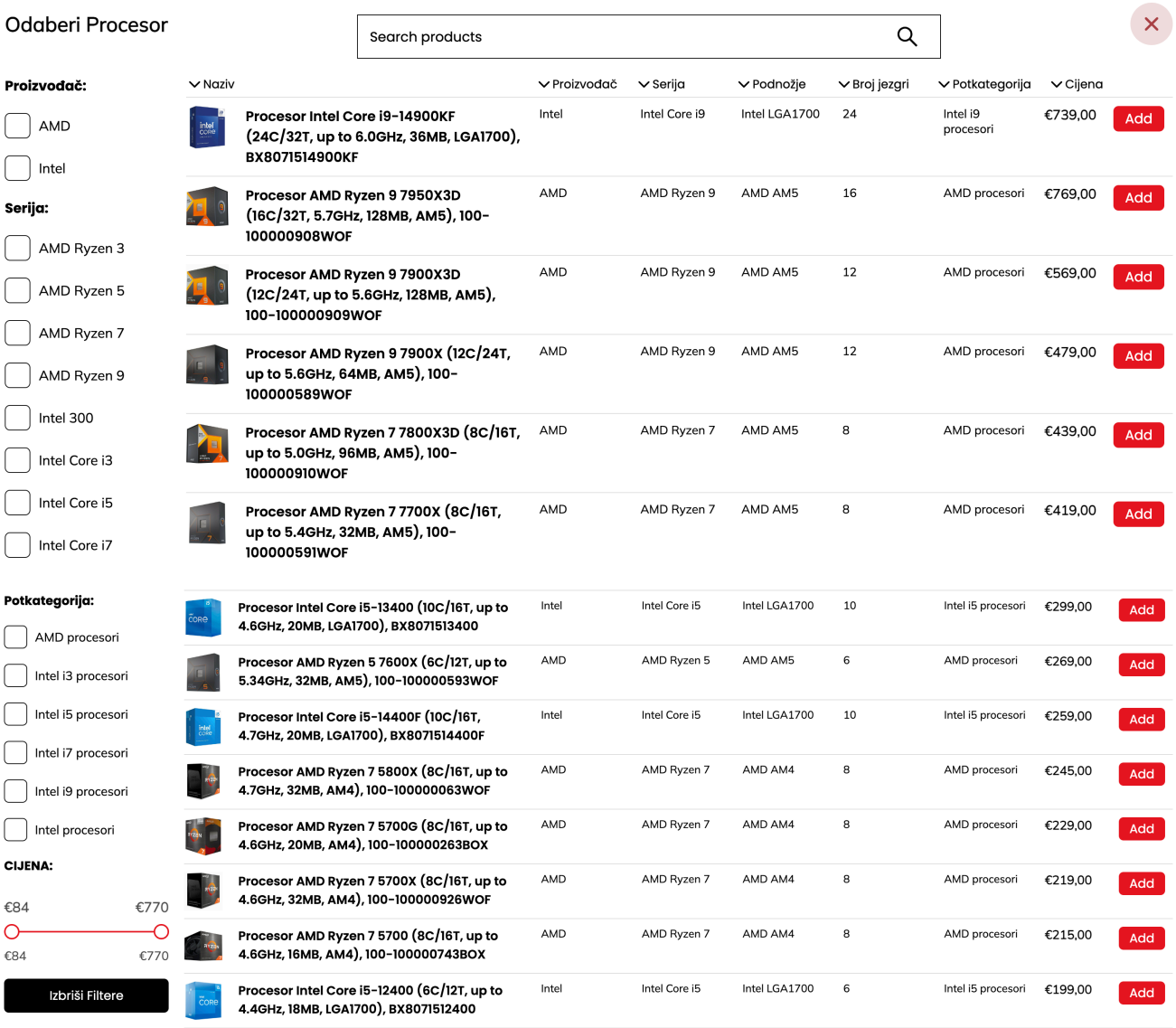
Klikom na poveznicu `Home` korisnika se preusmjerava na početnu stranicu aplikacije.

Klikom na poveznicu `Konfigurator` korisnika se preusmjerava na stranicu konfiguratora gdje korisnik može složiti konfiguraciju računala po želji i dodati dodatnu periferiju koja mu je potrebna te na kraju dodati cijelu konfiguraciju u košaricu. Izgled konfiguratora prikazan je na slici 11.




**Slika 11:** Izgled konfiguratora

Konfigurator je podijeljen u različite kategorije koje su potrebne za uspješno slaganje konfiguracije. Klikom na naziv kategorije korisnika se preusmjerava na stranicu za prikaz proizvoda u toj kategoriji. Klikom na botun `Odaberi <ime kategorije>` se otvara stranica koji sadrži proizvode u određenoj kategoriji među kojima korisnik bira proizvod koji želi dodati u konfiguraciju. Primjer otvorene stranice za odabir procesora prikazan je na slici 12. Klikom na botun `Clear Configuration` se briše cijela konfiguracija. Cijela konfiguracija se dodaje u korisnikovu košaricu klikom na botun `Kupi konfiguraciju`.




**Slika 12:** Stranica za odabir procesora

Korisnik može filtrirati proizvode po različitim ponuđenim filterima koji se nalaze s lijeve strane te pretraživati proizvode po nazivu proizvoda koristeći polje za pretraživanje (engl. *search field*) koje se nalazi na vrhu stranice. Klikom na neki od naziva potkategorija na vrhu tablice s proizvodima omogućava sortiranje podataka po odabranoj potkategoriji u silaznom ili uzlaznom smjeru. Smjer po kojem se proizvodi trenutno sortiraju označen je strelicom koja se nalazi pokraj naziva potkategorije. Ako strelica pokazuje prema dolje proizvodi se sortiraju silazno, a ako pokazuje prema gore proizvodi se sortiraju uzlazno. Po zadanim postavkama proizvodi se ne sortiraju. Mijenjanjem vrijednosti klizača cijena (engl. *price slider*) dohvaćaju se proizvodi veći i jednaki cijeni s lijeve strane te manji i jednaki

cijeni s desne strane. Prema zadanim postavkama su te vrijednosti postavljene na proizvod s najmanjom cijenom i proizvod s najvećom cijenom.

Programski kôd u ispisu 3 zadužen je za filtriranje, sortiranje i dohvaćanje proizvoda u određenoj kategoriji. Tijekom inicijalnog otvaranja stranice šalje se zahtjev GET kojim se željeni parametri prosljeđuju u metodu koja vraća filtrirane proizvode. Također zahtjev GET će se poslati pri svakoj interakciji korisnika, npr. kad korisnik odabere filter ili upiše upit u polje za pretraživanje. Kategorija proizvoda dohvaća se iz URL adrese stranice.

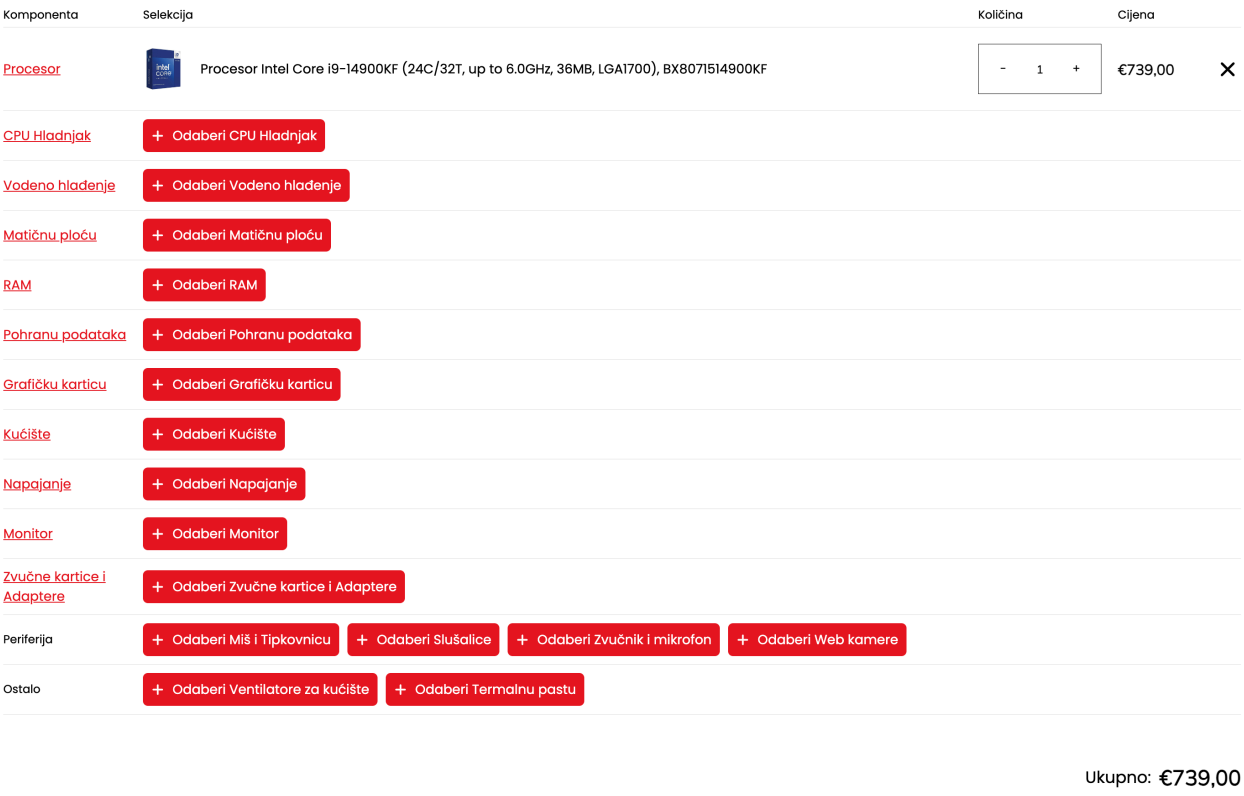
```
exports.getAllProductsByCategory = async (req, res) => {
     let { category } = req.params;
     let {
          sort,
          direction,
          page,
          pageSize,
          minPrice,
          maxPrice,
          activeFilters,
          search,
     } = req.query;
     if (sort == null && direction == null) {
          (sort = 'createdAt'), (direction = 'desc');
     }
     let query = {};
     if (minPrice && maxPrice) {
          if (search) {
               query = {
                    price: { $gte: minPrice, $lte: maxPrice },
                    category: category,
                    title: { $regex: search, $options: 'i' },
               };
          } else {
               query = {
                    price: { $gte: minPrice, $lte: maxPrice },
                    category: category,
               };
          }
     } else {
          query = { category: category };
     }
     if (activeFilters != null) {
          for (let activeFilter of activeFilters) {
               if (Object.values(activeFilter)[0] != '') {
                    query['details.' + Object.keys(activeFilter)] =
                         Object.values(activeFilter)[0];
               }
          }
     }
     let totalProducts;
     if (minPrice && maxPrice) {
          totalProducts = await Product.find(query).count();
     } else {
```

```
            totalProducts = await Product.find(query).count();
        }
        const totalPages = Math.ceil(totalProducts / pageSize);
        try {
            let products;
            products = await Product.find(query)
                .limit(pageSize)
                .skip(pageSize * page)
                .sort([[sort, direction]]);

            return res.status(200).send({
                success: true,
                data: products,
                totalProducts: totalProducts,
                totalPages: totalPages,
            });
        } catch (error) {
            console.log(error);
            return res.status(500).send({ success: false, error: error });
        }
};
```

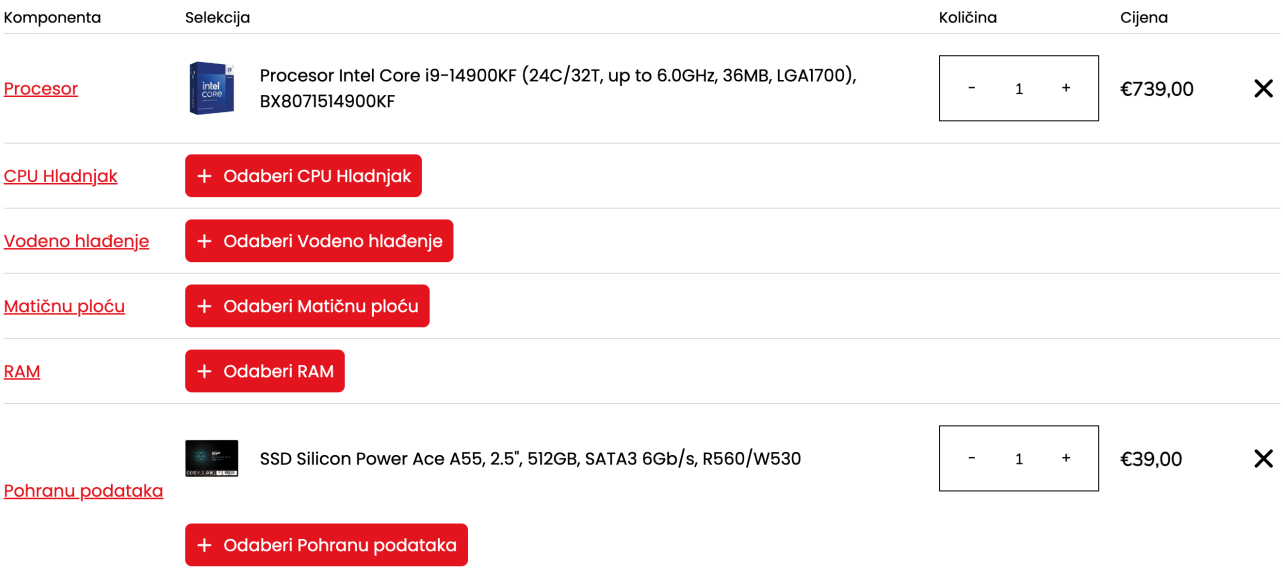
**Ispis 3:** Filtriranje, sortiranje i dohvaćanje proizvoda u određenoj kategoriji

Korisnik se može vratiti na prethodnu stranicu klikom na crveni x u gornjem desnom kutu. Klikom na botun `Add` proizvod se dodaje u korisnikovu konfiguraciju. Dodani proizvodi se nalaze u nizu koji se naziva `products` (hrv. proizvodi). Primjer prikaza konfiguratora s odabranim proizvodom nalazi se na slici 13.



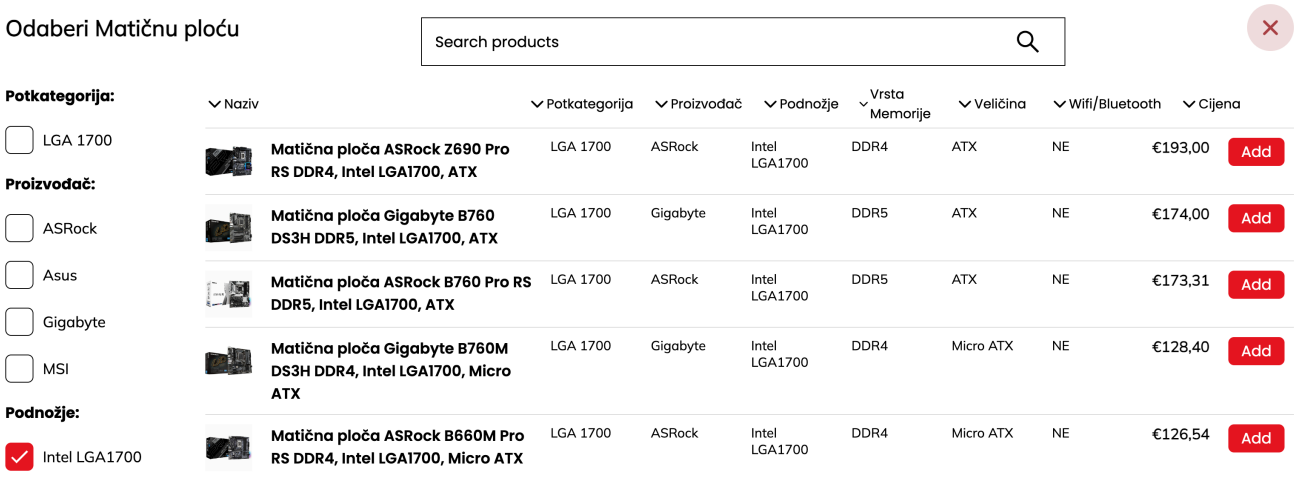

**Slika 13:** Konfigurator s odabranim procesorom

Na konfiguratoru se sada vidi drugačije korisničko sučelje. Botun za odabir procesora je skriven te je umjesto njega prikazana slika i naziv proizvoda koji je korisnik odabrao. S desne strane se nalazi polje za mijenjanje količine proizvoda kojem je inicijalna vrijednost 1. Korisnik naknadno može podešavati količinu proizvoda koristeći botune + i -. Kraj polja za mijenjanje količine nalazi se cijena proizvoda koja je pomnožena s odabranom količinom proizvoda. U ovom slučaju je količina 1 pa cijena označava cijenu jednog procesora. Kraj cijene se nalazi botun x koji omogućava brisanje proizvoda iz konfiguracije. Međutim postavlja se pitanje što ako korisnik želi dodati na primjer 2 SSD-a? U tom slučaju botun za dodatan odabir proizvoda se nalazi ispod odabranog proizvoda što je prikazano na slici 14.



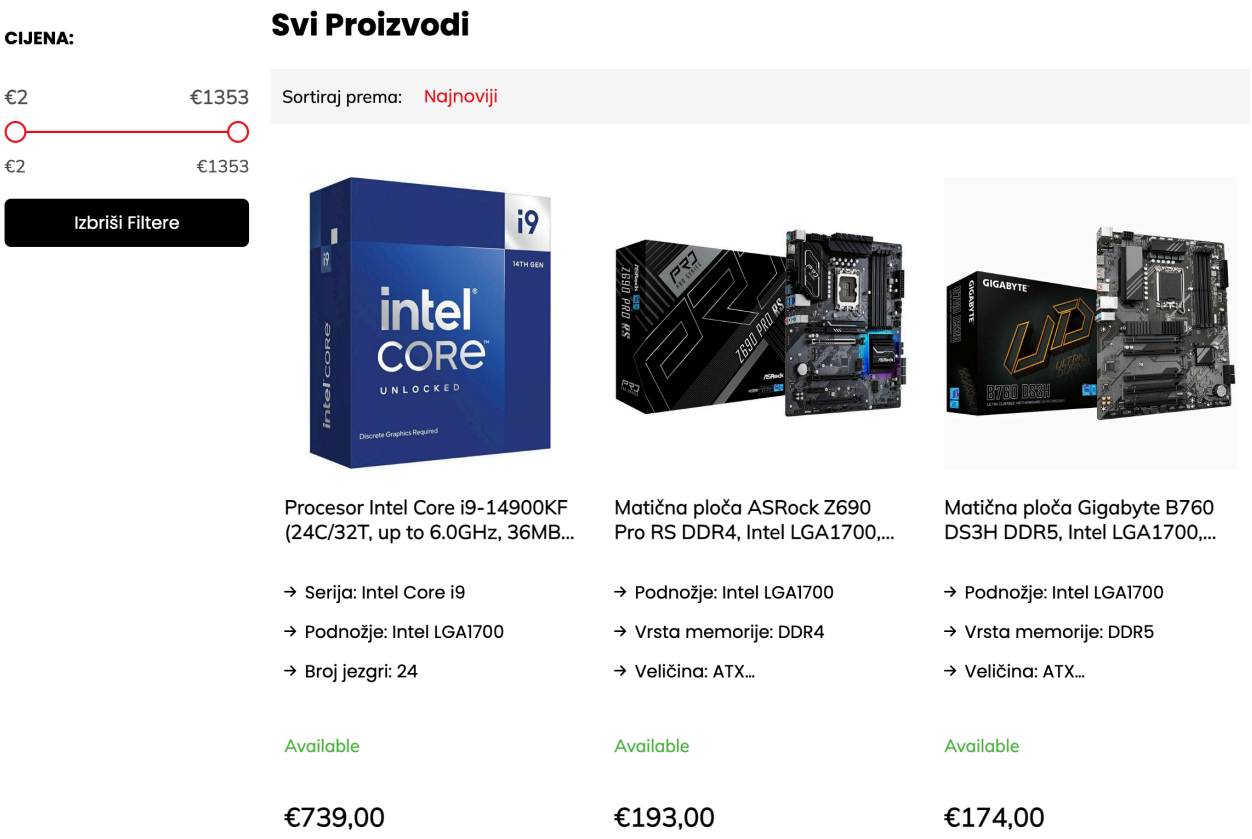

**Slika 14:** Konfigurator s odabranim procesorom i SSD-om

Nakon što korisnik u konfigurator odabere procesor, podnožje (engl. *socket*) od procesora će se spremiti u niz koji se naziva ograničenja (engl. *constraints*). Ako nakon toga želi odabrati matičnu ploču prikazat će se samo matične ploče koje su kompatibilne s odabranim procesorom na način da će se automatski odabrati filter za podnožje prilikom otvaranja stranice za odabir matične ploče, ako podnožje postoji u nizu ograničenja. Nakon odabira matične ploče podaci za odabir kućišta filtrirat će se na isti način što će omogućiti korisniku da odabere kućište u koje može stati odabrana matična ploča. Također prema odabranoj matičnoj ploči će se filtrirati RAM koji matična ploča podržava. Primjer filtriranja matične ploče po odabranom procesoru prikazan je na slici 15.




**Slika 15:** Primjer konfiguratora za odabir matične ploče nakon odabira procesora

Klikom na poveznicu `Svi proizvodi` korisnika se preusmjerava na stranicu za prikaz svih proizvoda dostupnih u aplikaciji. Izgled stranice za prikaz svih proizvoda prikazan je na slici 16.




**Slika 16:** Izgled stranice za prikaz svih proizvoda

Programski kôd u ispisu 4 zadužen je za sortiranje i dohvaćanje svih proizvoda na stranici korištenjem zahtjeva GET. Zahtjevi GET se šalju inicijalno i nakon svake interakcije korisnika s klizačem cijene i poljem za sortiranje na isti način kao i u konfiguratoru. Polje `sortiraj prema` na klik otvara padajući izbornik (engl. *dropdown*) koji korisniku nudi 5 opcija sortiranja: `Najnoviji`, `Naziv A-Z`, `Naziv Z-A`, `Cijena rastuća` i `Cijena padajuća`. Inicijalna opcija sortiranja je `Najnoviji` koja dohvaća najnovije proizvode.

```
exports.getAllProducts = async (req, res) => {
     let { sort, direction, page, pageSize, minPrice, maxPrice } =
req.query;
     if (sort == null && direction == null) {
          (sort = 'createdAt'), (direction = 'desc');
     }
     let totalProducts = await getTotalProducts(null, minPrice,
maxPrice);
     const totalPages = Math.ceil(totalProducts / pageSize);
     try {
          let products;
          products = await Product.find({
               price: { $gte: minPrice, $lte: maxPrice },
          })
               .limit(pageSize)
               .skip(pageSize * page)
               .sort([[sort, direction]]);

          return res.status(200).send({
               success: true,
               data: products,
               totalProducts: totalProducts,
               totalPages: totalPages,
          });
     } catch (error) {
          console.log(error);
          return res.status(500).send({ success: false, error: error });
     }
};
```

**Ispis 4:** Dohvaćanje svih proizvoda na stranici

Zbog mogućnosti da sustav sadrži velike količine proizvoda, prikaz proizvoda po stranici je ograničen na 12. Stranica na dnu sadrži botun `Prikaži više` s kojim se učitavaju dodatni proizvodi. Botun `Prikaži više` također prikazuje trenutni broj stranice i zadnji broj stranice koji postoji. Botun `Prikaži više` je prikazan na slici 17.




**Slika 17:** Izgled botuna prikaži više

Kad korisnik pronađe proizvod koji mu se sviđa, klikom na njega može pristupiti stranici na kojoj se nalaze detaljni podaci o proizvodu i opcije dodavanja proizvoda u listu želja i košaricu. Ako proizvod nije na stanju, botuni za dodavanje proizvoda u listu želja i dodavanje proizvoda u košaricu će biti onemogućeni. Izgled stranice prikazan je na slici 18.




**Slika 18:** Izgled stranice za prikaz podataka o proizvodu

Klikom na botun za dodavanje proizvoda u listu želja izvršit će se zahtjev POST i proizvod će se dodati u korisnikovu listu želja. Programski kôd u ispisu 5 prikazuje zahtjev POST za dodavanje proizvoda u korisnikovu listu želja.

```
exports.addToWishlist = async (req, res) => {
     const { userId, productId } = req.body;
     const product = await Product.findById(productId);
     const existingWishlist = await Wishlist.findOne({ userId: userId });
     if (!existingWishlist) {
          const wishlist = new Wishlist({
               userId,
               products: [product],
          });
          try {
               const savedWishlist = await wishlist.save();
               return res.status(200).send({ success: true, data:
savedWishlist });
          } catch (error) {
               return res.status(400).send({ success: false, error: error
});
          }
     } else {
          try {
```

```
                await existingWishlist.updateOne({ $push: { products:
product } });
                return res
                     .status(200)
                     .send({ success: true, data: 'Product added to
wishlist.' });
           } catch (error) {
                return res.status(400).send({ success: false, error: error
});
           }
     }
};
```

**Ispis 5:** Dodavanje proizvoda u listu želja

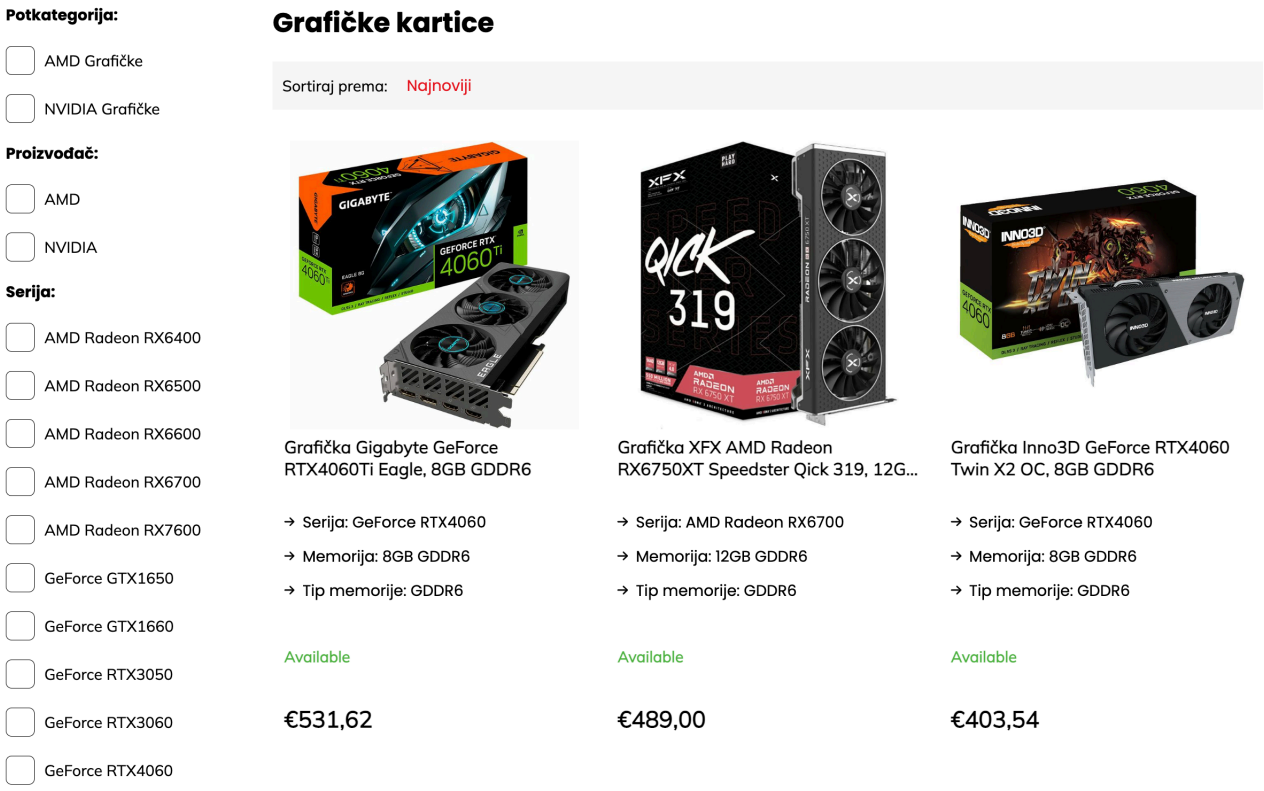
Klikom na botun `Add to cart` neće se izvršiti nikakav zahtjev već će se proizvod dodati u niz pod nazivom `products` ako se ne nalazi u košarici, a ako se nalazi u košarici povećat će se količina tog proizvoda u košarici za 1. Cijeli proces dodavanja proizvoda u košaricu prikazan je u ispisu 6.

```
          addProduct: (state, action) => {
                let productInCartId = null;
                for (let i = 0; i < state.products.length; i++) {
                     if (state.products[i]._id === action.payload._id) {
                          productInCartId = i;
                     }
                }
                if (productInCartId != null) {
                     state.products[productInCartId].quantity +=
action.payload.quantity;
                     state.quantity += action.payload.quantity;
                     let priceToAdd = action.payload.price *
action.payload.quantity;
                     let tempCartTotal =
                          Number.parseFloat(state.totalPrice) +
Number.parseFloat(priceToAdd);
                     state.totalPrice = tempCartTotal;
                } else {
                     state.quantity += action.payload.quantity;
                     state.products.push(action.payload);
                     let priceToAdd = action.payload.price *
action.payload.quantity;
                     let tempCartTotal =
                          Number.parseFloat(state.totalPrice) +
Number.parseFloat(priceToAdd);
                     state.totalPrice = tempCartTotal;
                }
          },
```

**Ispis 6:** Dodavanje proizvoda u košaricu

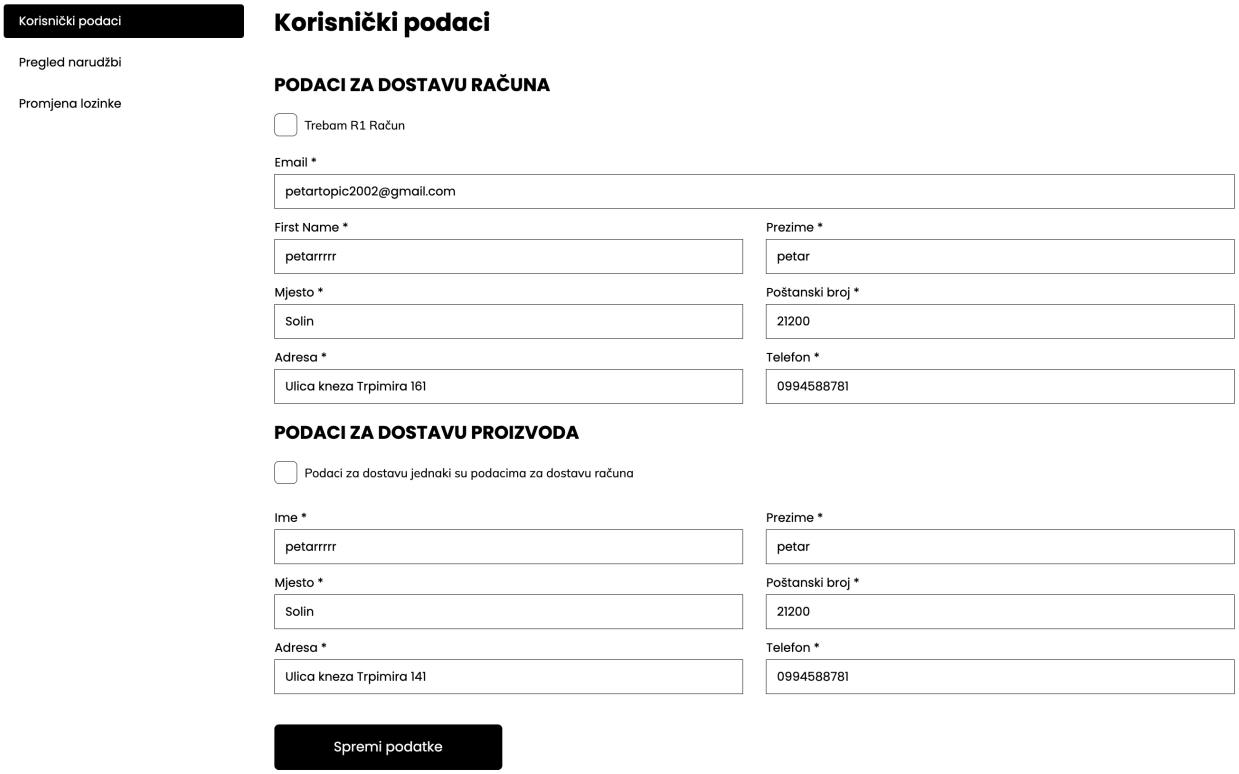
Klikom na neku od ponuđenih kategorija, korisnika se preusmjerava na stranicu za prikaz proizvoda u kategoriji koju je odabrao. Izgled stranice je gotovo identičan onoj za

prikaz svih proizvoda, samo što se s lijeve strane nalaze filteri za odabranu kategoriju proizvoda. Dohvaćanje proizvoda po određenoj kategoriji se obavlja s istim zahtjevom GET koji je korišten u konfiguratoru. Izgled stranice prikazan je na slici 19.




**Slika 19:** Izgled stranice za prikaz proizvoda u određenoj kategoriji

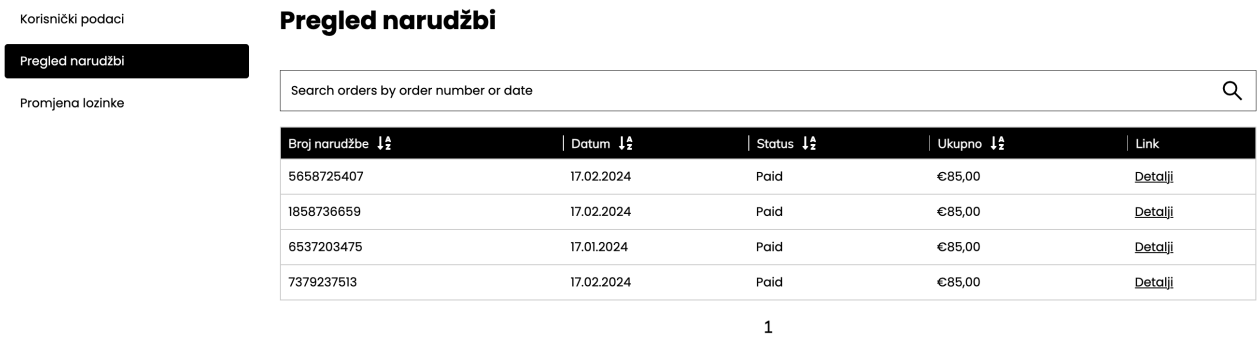
- Ikona za odjavu – Klikom na ikonu za odjavu će se odjaviti trenutno prijavljeni korisnik.
- Ikona korisnika – Klikom na ikonu korisnika otvara se stranica za upravljanje korisničkim podacima. Na toj stranici se nalaze podstranice koje korisniku omogućuju da uredi svoje podatke, vidi sve svoje narudžbe i promijeni svoju lozinku. U slučaju da korisnik nije prijavljen u sustav, preusmjerava ga se na formu za prijavu u sustav. Na podstranici `Korisnički podaci` prikazani su svi podaci korisnika. Izgled podstranice prikazan je na slici 20.




**Slika 20:** Izgled podstranice za prikaz i uređivanje podataka o korisniku

Korisnik može urediti podatke u poljima te ih spremiti klikom na botun `Spremi podatke`. Na stranici postoje 2 dodatna botuna `Trebam R1 račun` i `Podaci za dostavu jednaki su podacima za dostavu računa`. Klikom na botun `Trebam R1 račun` prikazuju se dva dodatna polja za unos naziva tvrtke i OIB-a tvrtke. Klikom na botun `Podaci za dostavu jednaki su podacima za dostavu računa` se skriva druga forma u koju se unose podaci za dostavu proizvoda te se u nju prekopiraju podaci iz prve forme za dostavu računa. Svi uneseni podaci će se koristiti za ubrzavanje procesa kupovine tako što će se automatski popuniti u koraku gdje korisnik treba unijeti podatke za dostavu računa i proizvoda koji će se pojasniti u nastavku rada u stavki košarice.

Na podstranici `Pregled narudžbi` nalaze se sve korisnikove narudžbe prikazane u tablici. Izgled stranice prikazan je na slici 21.




**Slika 21:** Podstranica za prikaz narudžbi

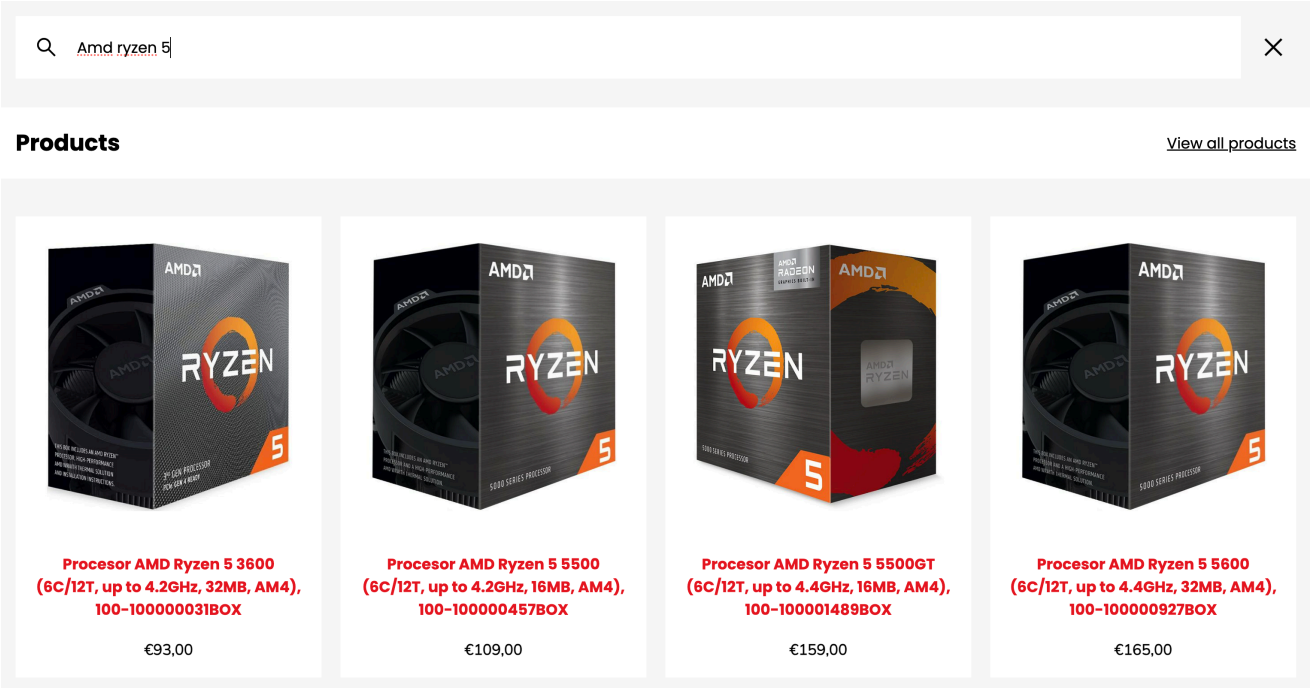
Zadnja podstranica koristi se za promjenu korisnikove lozinke. Podstranica sadrži dva polja. Jedno polje se odnosi na korisnikovu trenutnu lozinku, a drugo na novu lozinku s kojom korisnik želi zamijeniti staru lozinku. Izgled stranice prikazan je na slici 22.




**Slika 22:** Podstranica za promijenu lozinke

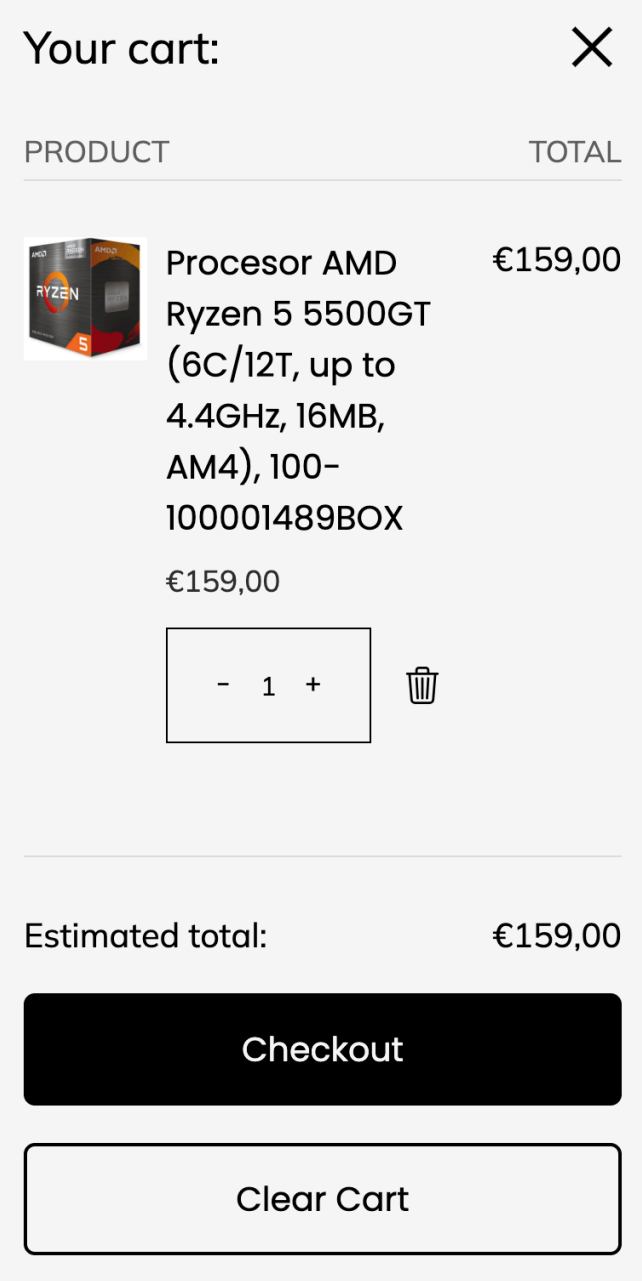
Lozinka se mijenja klikom na botun `Reset password` nakon unosa svih potrebnih podataka u formu. U slučaju da korisnik unese pogrešnu trenutnu lozinku prikazat će se greška te se trenutna lozinka neće promijeniti.

- Ikona za pretraživanje proizvoda – Klikom na ikonu za pretraživanje proizvoda otvara se stranica za pretraživanje proizvoda. Stranica sadrži polje za pretraživanje i botun za zatvaranje stranice. Izgled stranice prikazan je na slici 23.




**Slika 23:** Stranica za pretraživanje proizvoda

- Ikona za listu želja – Klikom na ikonu za listu želja, korisnika se preusmjerava na stranicu za prikaz njegove liste želja. Ako korisnik nije prijavljen u sustav prikazat će mu se stranica s formom za prijavljivanje u sustav.
- Ikona za košaricu – Klikom na ikonu košarice otvara se modal s desne strane koji prikazuje proizvode koje je korisnik dodao u košaricu. U slučaju da korisnik nije dodao niti jedan proizvod u košaricu prikazat će se tekst `Your cart is empty` i botun `Continue shopping` koji zatvara modal. Ako je korisnik dodao proizvode u košaricu može ih vidjeti u modalu te upravljati s njima. Korisnik može povećati ili smanjiti količinu određenog proizvoda u košarici za jedan klikom na botune + ili -. Također može izbrisati proizvod iz košarice klikom na ikonicu kante za smeće. Navedene opcije prikazane su na slici 24.




**Slika 24:** Prikaz košarice

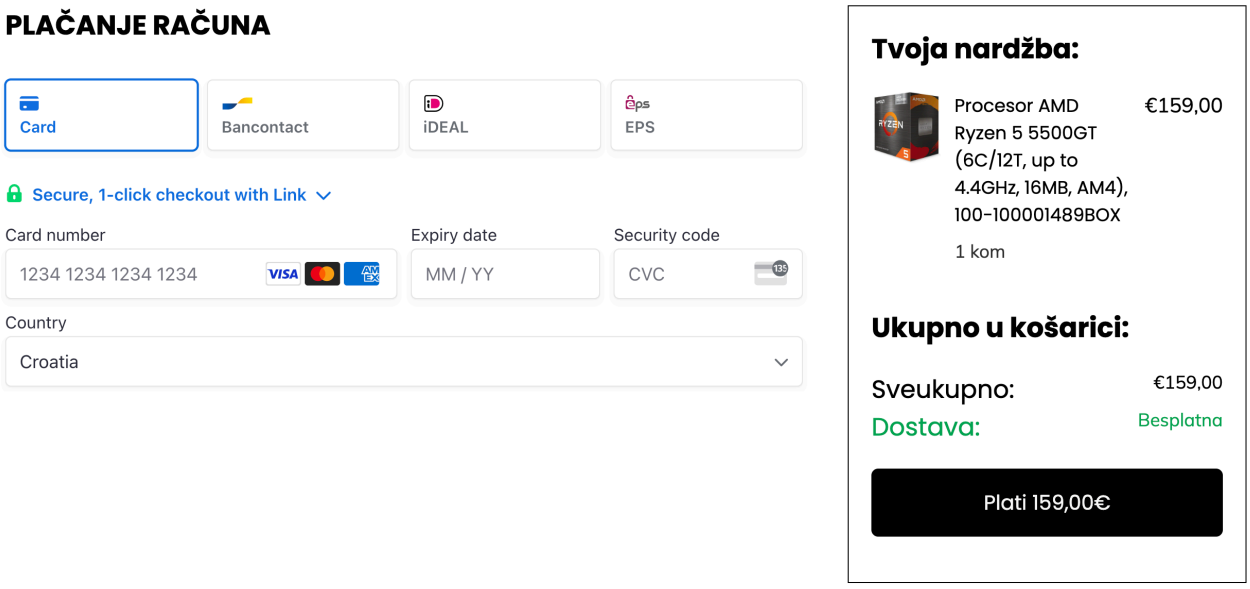
Programski kôd zaslužan za upravljanje proizvodima u košarici prikazan je u dodatku 2.

Kad korisnik odluči kupiti proizvode u svojoj košarici. Klikom na botun `Checkout` preusmjerit će se na stranicu za unošenje podataka za dostavu računa i dostavu proizvoda. Izgled stranice prikazan je na slici 25.




**Slika 25:** Stranica za unos podataka o dostavi računa i proizvoda

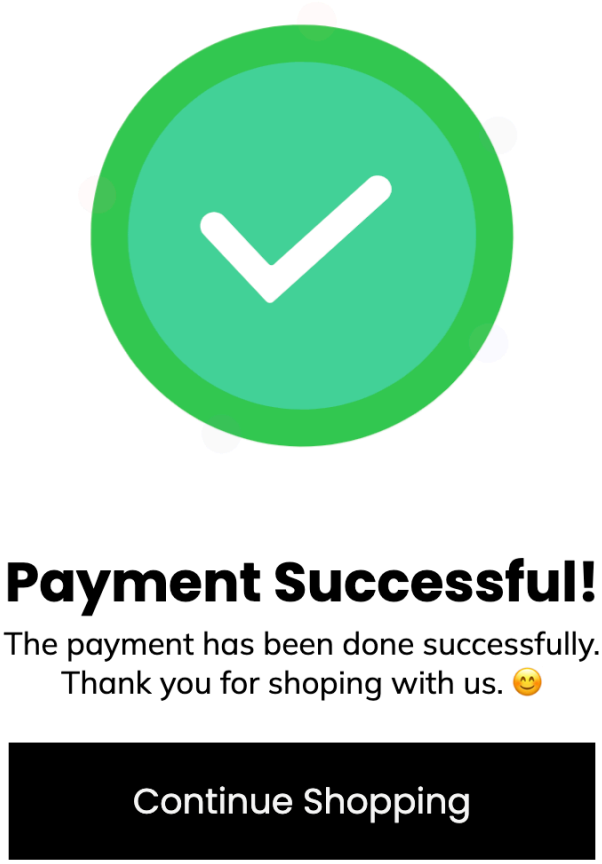
Kao što je već spomenuto, podaci će se automatski popuniti ako ih je korisnik unio u formu za uređivanje korisničkih podataka. Klikom na botun `Idi na plaćanje` korisnika se preusmjerava na stranicu za izvršavanje transakcije koja koristi paket Stripe za validaciju podataka o kreditnoj kartici te izvršavanje plaćanja. Izgled stranice za izvršavanje transakcije prikazan je na slici 26.




**Slika 26:** Stranica za izvršavanje transakcije

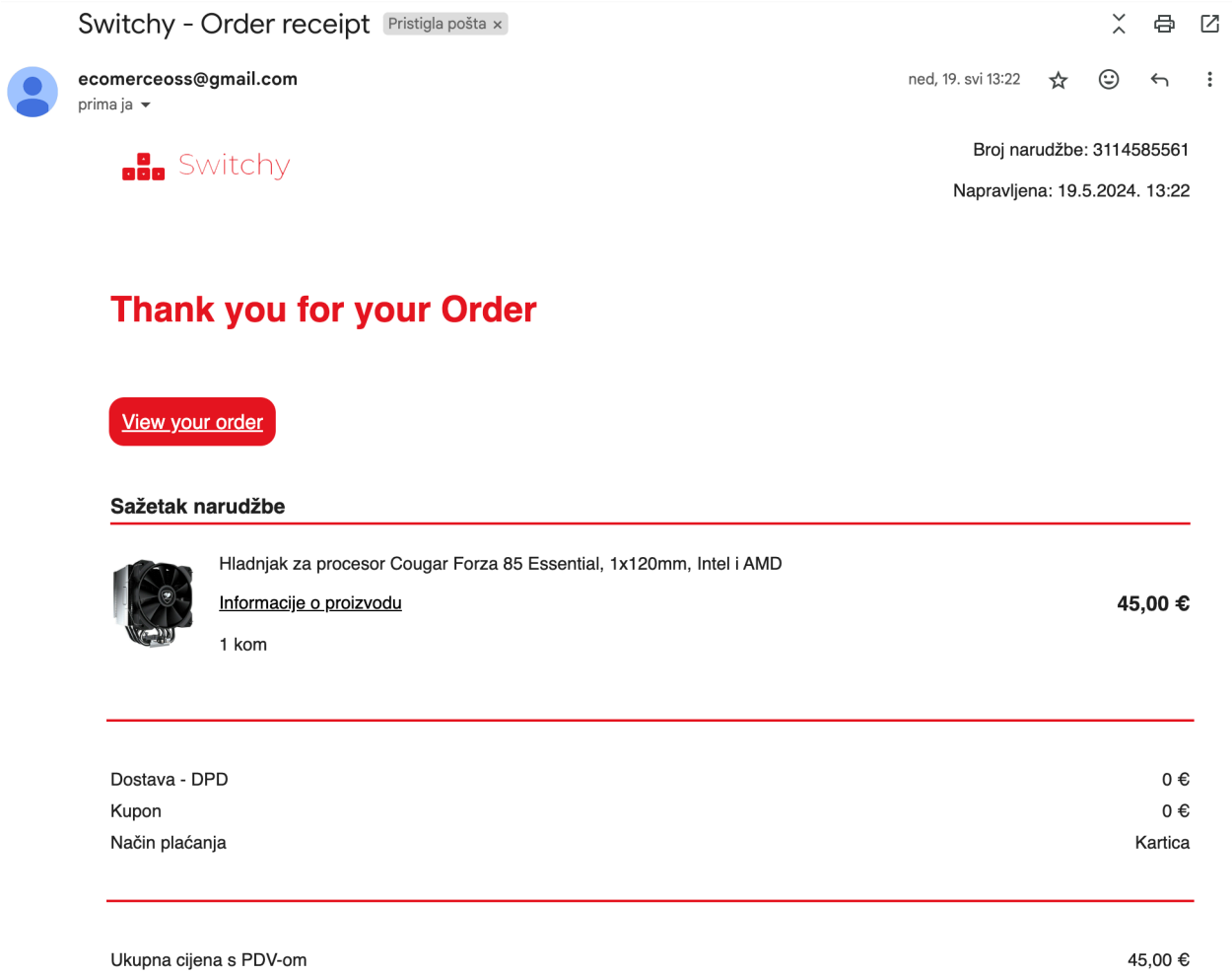
Dodatak 3 prikazuje programski kôd zaslužan za obrađivanje transakcije koristeći paket Stripe.

Nakon uspješne transakcije korisnika će se preusmjeriti na stranicu s porukom o uspješnoj kupnji prikazanoj na slici 27.



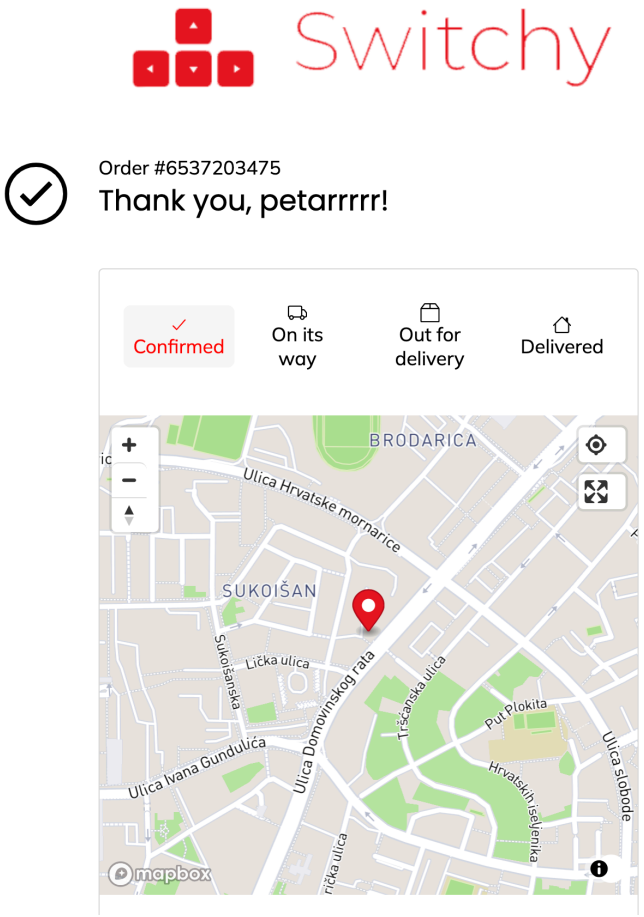

**Slika 27:** Stranica s porukom o uspješnoj kupnji

Korisniku će se u međuvremenu poslati poruka o narudžbi na adresu elektronske pošte koju je unio u prethodnom koraku gdje je unosio podatke za dostavu računa. Primjer poruke prikazan je na slici 28.




**Slika 28:** Primjer poruke o narudžbi

Klikom na botun `View your order` otvara se stranica za pregled narudžbe u novom prozoru web preglednika. Izgled stranice prikazan je na slici 29. Na stranici se nalaze informacije za dostavu i informacije o naručenim proizvodima. Stranica također sadrži kartu na kojoj se nalazi marker koji označava korisnikovu unesenu adresu za dostavu proizvoda. Poviše karte se nalaze 4 kartice koje označavaju trenutni status narudžbe. Prva kartica će biti crvene boje ako je narudžba plaćena, druga ako se narudžba spremna za dostavu, treća ako je narudžba na dostavi te zadnja ako je narudžba dostavljena.




1 **Procesor AMD Ryzen 3 4100 (4C/8T, up to 4.0GHz, 4MB, AM4), 100-100000510BOX** **85,00€**

| | |
|---|---|
| Subtotal | **85,00€** |
| Discount | **0.00€** |
| Shipping | **0.00€** |
| Total | 85,00€ |

## Order details

**Contact information**
petartopic2002@gmail.com

**Shipping address**
petarrrrr petar
Ulica kneza Trpimira 141
21208 Solin
Croatia
0994588781

**Billing address**
petarrrrr petar
Ulica kneza Trpimira 141
21208 Solin
Croatia

Need help? Contact Us

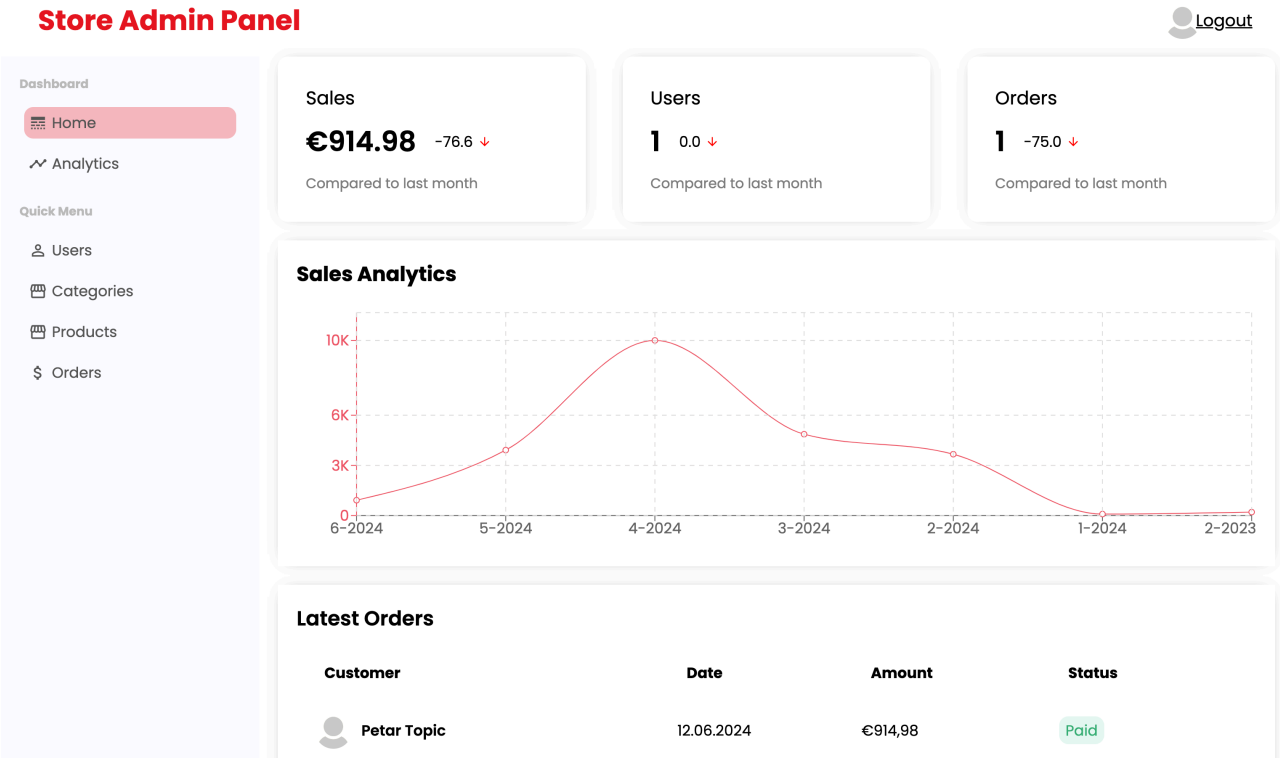
Continue shopping

**Slika 29:** Izgled stranice za pregled narudžbe

### 3.2.2. Aplikacija namijenjena za administratore sustava

Aplikacija namijenjena za administratore sustava koristi se u svrhe pregleda uspjeha trgovine koristeći određene analitičke grafove koji prikazuju promet ostvaren od prodaje proizvoda, broj novih korisnika koji su se registrirali u sustav i broj narudžbi. Također administratoru je omogućeno upravljanje svim korisnicima, proizvodima, kategorijama proizvoda i narudžbama koje se nalaze u trgovini na način da može dodavati nove podatke u trgovinu, uređivati podatke koji se nalaze u trgovinu te brisati podatke iz trgovine.

Nakon uspješne prijave u aplikaciju za administratore sustava prikazuje se glavna stranica aplikacije prikazana na slici 30. Glavna stranica sadrži izbornik s lijeve strane te sadržaj s desne strane. U gornjem desnom kutu se nalazi botun za odjavljivanje korisnika iz sustava.




**Slika 30:** Prikaz glavne stranice aplikacije za administratore

Na vrhu glavne stranice nalaze se 3 kartice koje prikazuju neke od najbitnijih statistika trgovine. Prva kartica prikazuje količinu novca ostvarenog od prodaje proizvoda za trenutni mjesec u godini. Pokraj količine se nalazi postotak koji prikazuje rast ili pad s obzirom na vrijednost od prošlog mjeseca. Druga kartica prikazuje broj novih korisnika koji su se

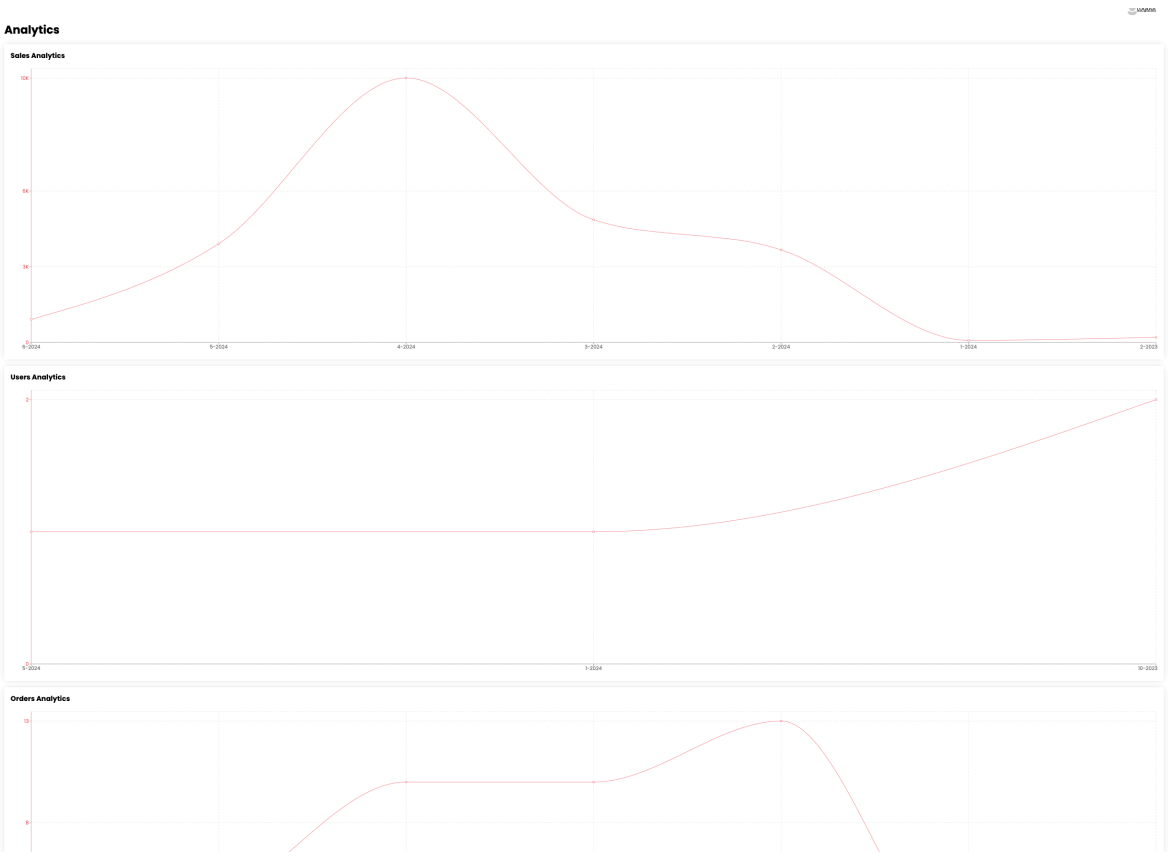
registrirali u trgovinu. Treća kartica prikazuje broj narudžbi koje su napravljene od strane korisnika.

Ispod kartica se nalazi graf koji prikazuje promet ostvaren od prodaje proizvoda grupiran po mjesecima. Promet za trenutni mjesec se nalazi s lijeve strane te se udaljavaju prema desnoj strani za svaki mjesec koji prethodi trenutnom i koji sadrži podatke.

Na dnu stranice se nalazi prikaz najnovijih narudžbi. Prikazuju se 4 najnovije narudžbe koje se dohvaćaju iz tablice `orders` prema polju `createdAt` sortirano u silaznom smjeru.

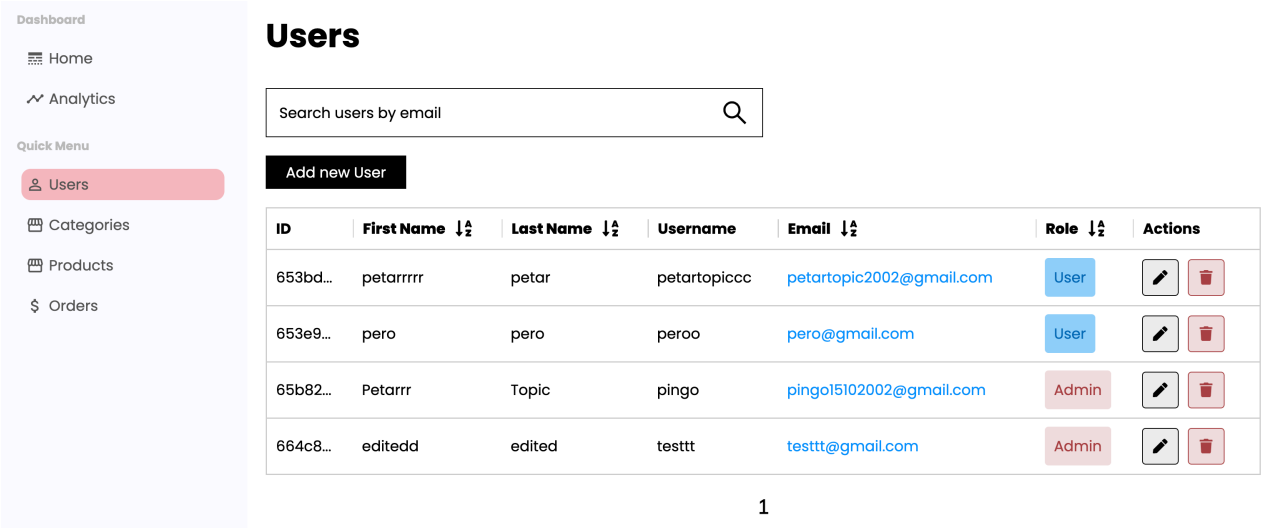
Iduća stranica u izborniku je stranica za prikaz analitike trgovine. Na stranici se nalaze 3 grafa za detaljniji prikaz informacija koje su se nalazile u karticama na vrhu prethodne stranice. Izgled stranice prikazan je na slici 31.

Na vrhu se nalazi graf koji je identičan onome koji se nalazio na prethodnoj stranici koji prikazuje promet ostvaren od prodaje proizvoda u obliku grafa grupiranog po mjesecima. Ispod njega se nalazi graf koji na isti način prikazuje podatke o broju registriranih korisnika u trgovini grupirano po mjesecima. Na zadnjem grafu se prikazuju podaci o broju narudžbi u trgovini grupirano po mjesecima.



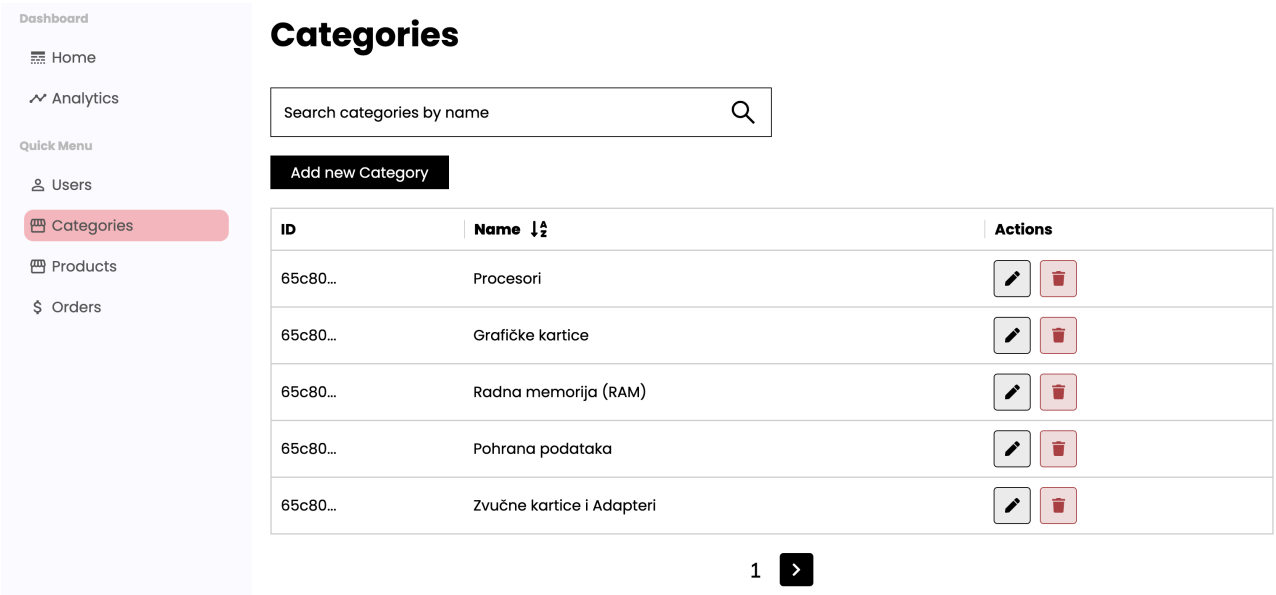

**Slika 31:** Izgled stranice za prikaz analitike trgovine

Iduća stranica u izborniku je stranica za prikaz svih korisnika u sustavu. Na njoj se nalazi tablica koja prikazuje sve korisnike u sustavu. Prikaz je ograničen na 5 elemenata po stranici te se stranice mijenjaju s paginacijom koja se nalazi ispod tablice. Na vrhu tablice se nalazi polje za pretraživanje i botun za dodavanje novog korisnika koji otvara formu za unošenje podataka o novom korisniku. Tablica funkcionira na istom principu kao ona u konfiguratoru. Jedina razlika je što se u tablici za prikaz korisnika u zadnjem stupcu nalaze botuni za uređivanje podataka o korisniku i brisanje korisnika. Izgled stranice za prikaz svih korisnika u sustavu nalazi se na slici 32.



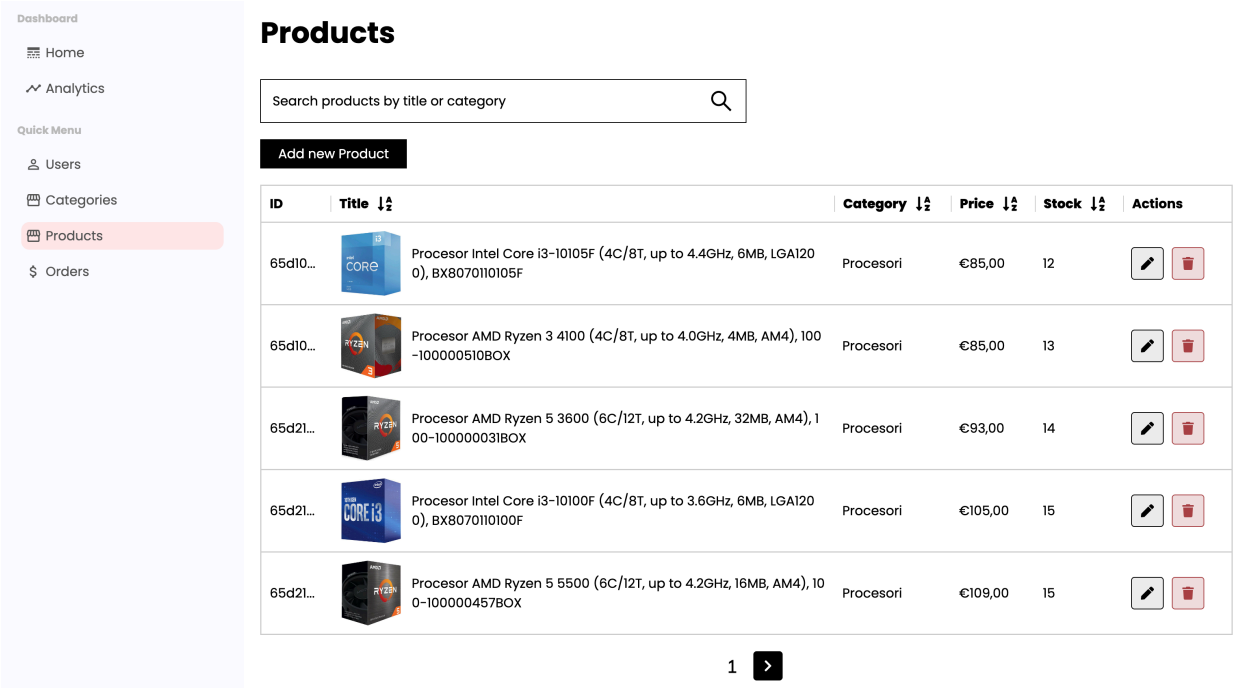

**Slika 32:** Izgled stranice za prikaz svih korisnika u sustavu

Iduća stranica u izborniku je stranica za prikaz svih kategorija u sustavu. Stranica je napravljena na isti način kao prethodna stranica za prikaz svih korisnika u sustavu. Sadrži polje za pretraživanje, botun za dodavanje nove kategorije i tablicu s akcijama i paginacijom. Izgled stranice prikazan je na slici 33.



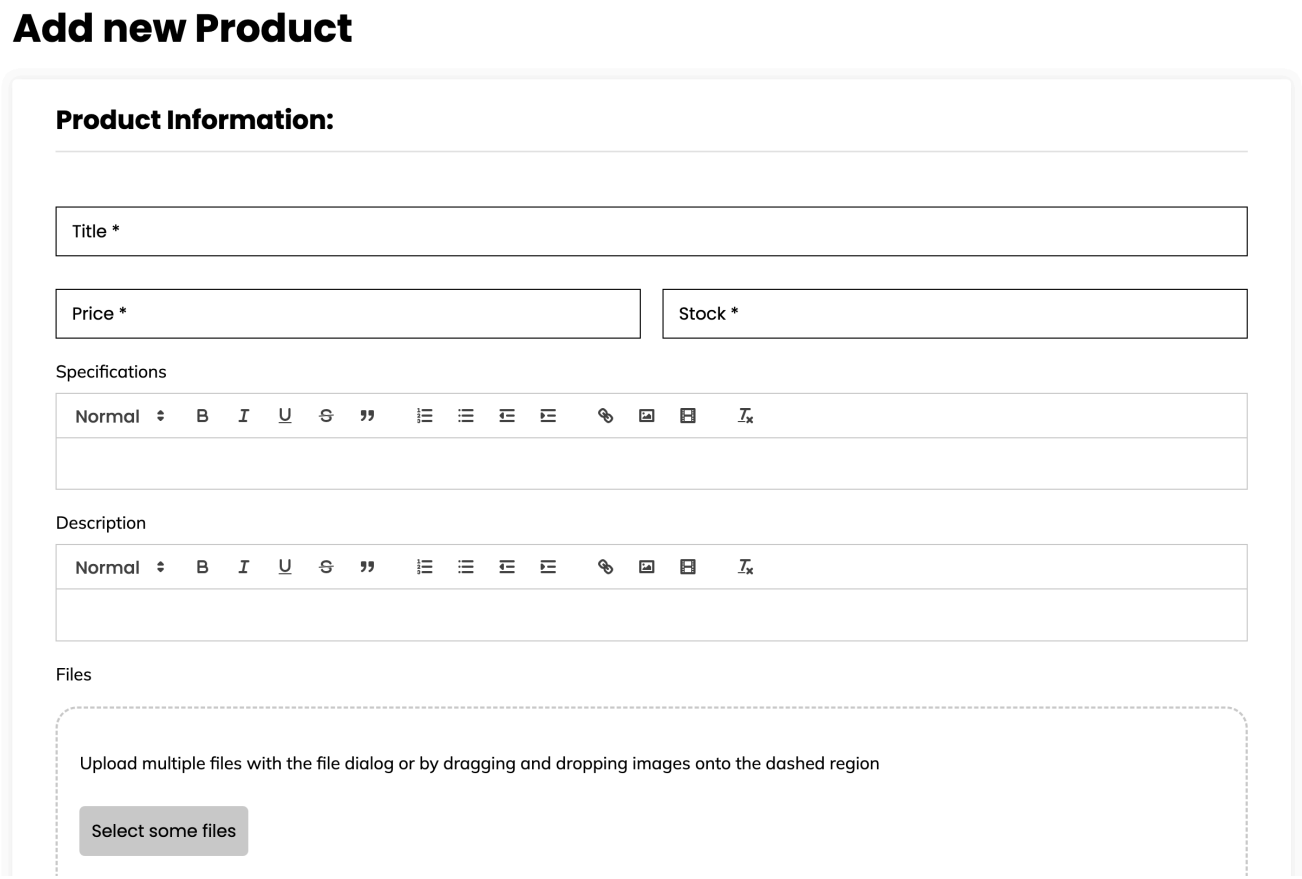

**Slika 33:** Izgled stranice za prikaz svih kategorija u sustavu

Iduća stavka je stranica za prikaz svih proizvoda u sustavu. Korisničko sučelje je slično kao na prethodnim stranicama. Izgled korisničkog sučelja prikazan je na slici 34.




**Slika 34:** Izgled korisničkog sučelja za prikaz svih proizvoda u sustavu

Jedina razlika u korisničkom sučelju je prilikom dodavanja novog proizvoda i uređivanja postojećeg proizvoda. Izgled stranice za dodavanje novog proizvoda prikazan je na slici 35.




**Slika 35:** Forma za dodavanje novog proizvoda u sustav

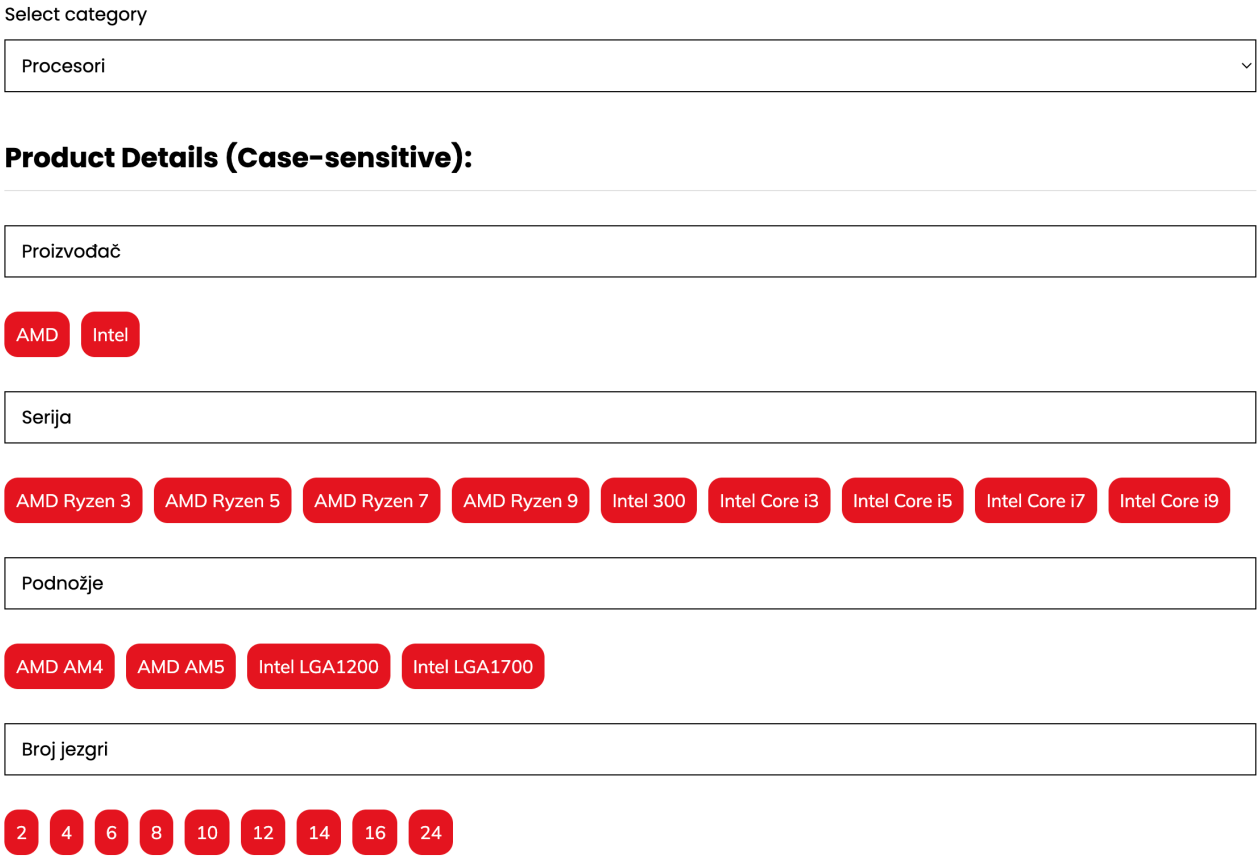
Dodana su 3 nova tipa polja za potrebe dodavanja proizvoda u sustav. Jedno od takvih polja jest polje za unos opisa proizvoda koje ima razne opcije za formatiranje i pisanje teksta u svrhu ljepšeg prikazivanja opisa proizvoda. Drugo polje koje je dodano je polje za dodavanje slika koje će se prikazati na stranici za prikaz detalja proizvoda. Slike se pohranjuju (engl. *upload*) na oblak (engl. *cloud*) koristeći servis cloudinary. Za pohranjivanje slika pomoću POST zahtejva koristi se paket cloudinary. Korištenjem cloudinary paketa pohranjene slike se vraćaju kao URL koji se sprema u niz stringova koji predstavlja slike za određeni proizvod. U ispisu 7 prikazana je funkcija za pohranjivanje slika.

```
exports.uploadToCloudinary = (path, folder) => {
     return cloudinary.v2.uploader
          .upload(path, {
               folder,
          })
          .then((data) => {
               return { url: data.url };
          })
          .catch((error) => {
               console.log(error);
          });
};
```

**Ispis 7:** Funkcija za pohranjivanje slika koristeći cloudinary paket

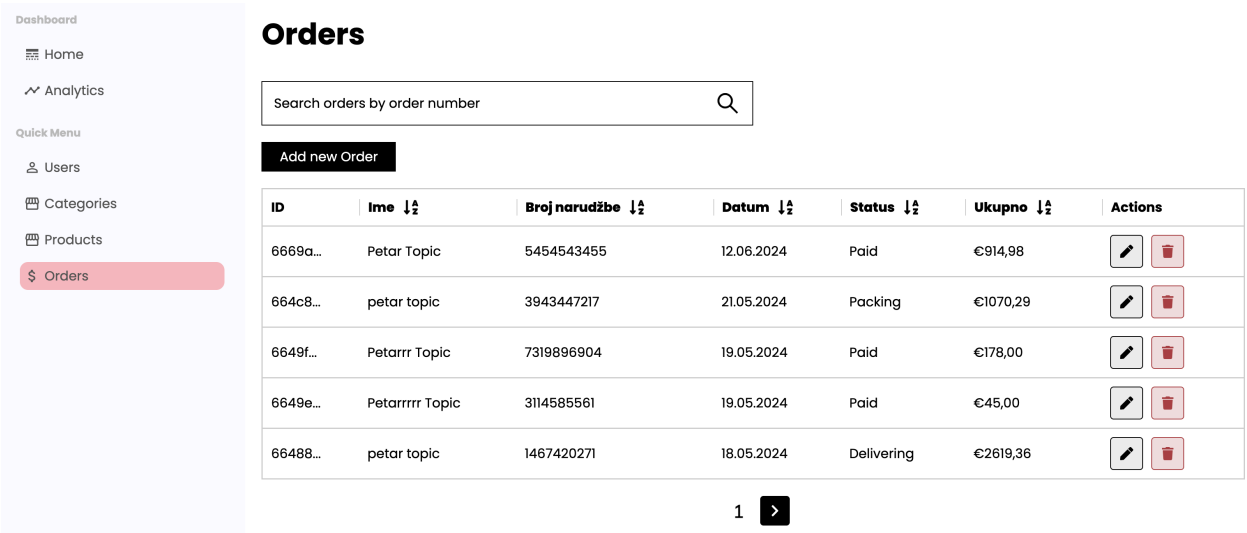
Na stranici gdje se prikazuju svi proizvodi ili proizvodi određene kategorije koristit će se slika koja je pohranjena kao prva slika proizvoda.

Zadnji novi tip polja su polja koja se prikazuju kad administrator odabere kategoriju proizvoda iz padajućeg izbornika. Nakon odabira kategorije prikazat će se polja za unos filtera za odabranu kategoriju. Ispod polja gdje se može unijeti tekst nalaze se svi filteri koji su do sada dodani u sustav za odabranu kategoriju proizvoda. Prikazuju se samo jedinstvene vrijednosti. Klikom na određeni filter vrijednost se popunjava u polje za unos filtera. Primjer za kategoriju procesori prikazan je na slici 36.




**Slika 36:** Prikaz sučelja za dodavanje novog proizvoda nakon odabira kategorije proizvoda

Zadnja stavka u izborniku je stranica za prikaz svih narudžbi u sustavu. Korisničko sučelje izgleda isto kao na prethodnim stranicama. Izgled korisničkog sučelja prikazan je na slici 37.




**Slika 37:** Korisničko sučelje za prikaz svih narudžbi u sustavu

# 4. Zaključak

Današnje web trgovine predstavljaju nezaobilazan segment modernog digitalnog poslovanja, te omogućuju učinkovitu prodaju proizvoda i usluga.

U okviru ovog rada razvijena je funkcionalna web trgovina za prodaju informatičke opreme, pomoću tehnologija kao što su HTML, CSS, JavaScript, React, MongoDB, Node.js i Express.js. Implementirane su osnovne značajke poput pregleda, pretraživanja i filtriranja proizvoda te upravljanja korisničkim podacima i narudžbama. Osim toga, integrirano je testno kartično plaćanje putem Stripe platforme, što osigurava sigurnost i pouzdanost sustava prije postavljanja na internet.

Rezultat ovog rada predstavlja temelj za uspostavu stvarnog online poslovanja putem razvijene web trgovine. Kroz primijenjene tehnologije i funkcionalnosti, korisnicima se omogućuje jednostavno pregledavanje i kupovina informatičke opreme, dok administrativno sučelje pruža učinkovito upravljanje sadržajem trgovine.

Prijedlozi za daljnji razvoj aplikacije uključuju omogućavanje dodavanja proizvoda različitih varijacija, poput crnih i bijelih slušalica, te prikaz tih varijacija na istoj stranici proizvoda. Trenutno je implementirano rješenje u kojem administrator mora dodati dva ista proizvoda s različitim slikama i nazivima boje, što nije idealno, posebno ako proizvodi imaju različite cijene. Također, predlaže se uvođenje kupona za sniženje cijene narudžbe, koje bi dodavao administrator.

# Literatura

[1] „HTML Introduction“, https://www.geeksforgeeks.org/html-introduction/ (posjećeno 25.8.2024)

[2] „CSS Introduction“, https://www.w3schools.com/css/css_intro.asp (posjećeno 25.8.2024)

[3] „How To Add CSS“, https://www.w3schools.com/css/css_howto.asp (posjećeno 25.8.2024)

[4] „Most popular technologies“, https://survey.stackoverflow.co/2024/technology#most-popular-technologies-language (posjećeno 25.8.2024)

[5] „What Can You Do with JavaScript?“, https://www.orientsoftware.com/blog/what-can-you-do-with-javascript/ (posjećeno 25.8.2024)

[6] „What is JavaScript?“, https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/What_is_JavaScript (posjećeno 25.8.2024)

[7] „Is JavaScript Interpreted or Compiled?“, https://www.geeksforgeeks.org/is-javascript-interpreted-or-compiled/ (posjećeno 25.8.2024)

[8] „Quick Start“, https://react.dev/learn (posjećeno 25.8.2024)

[9] „The Best Guide to Know What Is React“, https://www.simplilearn.com/tutorials/reactjs-tutorial/what-is-reactjs (posjećeno 25.8.2024)

[10] „MVC Framework Introduction“, https://www.geeksforgeeks.org/mvc-framework-introduction/ (posjećeno 25.8.2024)

[11] „Model View Controller architecture in Programming“, https://www.linkedin.com/pulse/model-view-controller-architecture-programming-aditira-jamhuri/ (posjećeno 7.9.2024)

[12] „MongoDB: An introduction“, https://www.geeksforgeeks.org/mongodb-an-introduction/ (posjećeno 25.8.2024)

[13] „MongoDB Pricing“, https://www.mongodb.com/pricing (posjećeno 25.8.2024)

[14] „Create Relationship in MongoDB“, https://www.geeksforgeeks.org/create-relationship-in-mongodb/ (posjećeno 25.8.2024)

[15] „Node.js Introduction“, https://www.geeksforgeeks.org/node-js-introduction/ (posjećeno 25.8.2024)

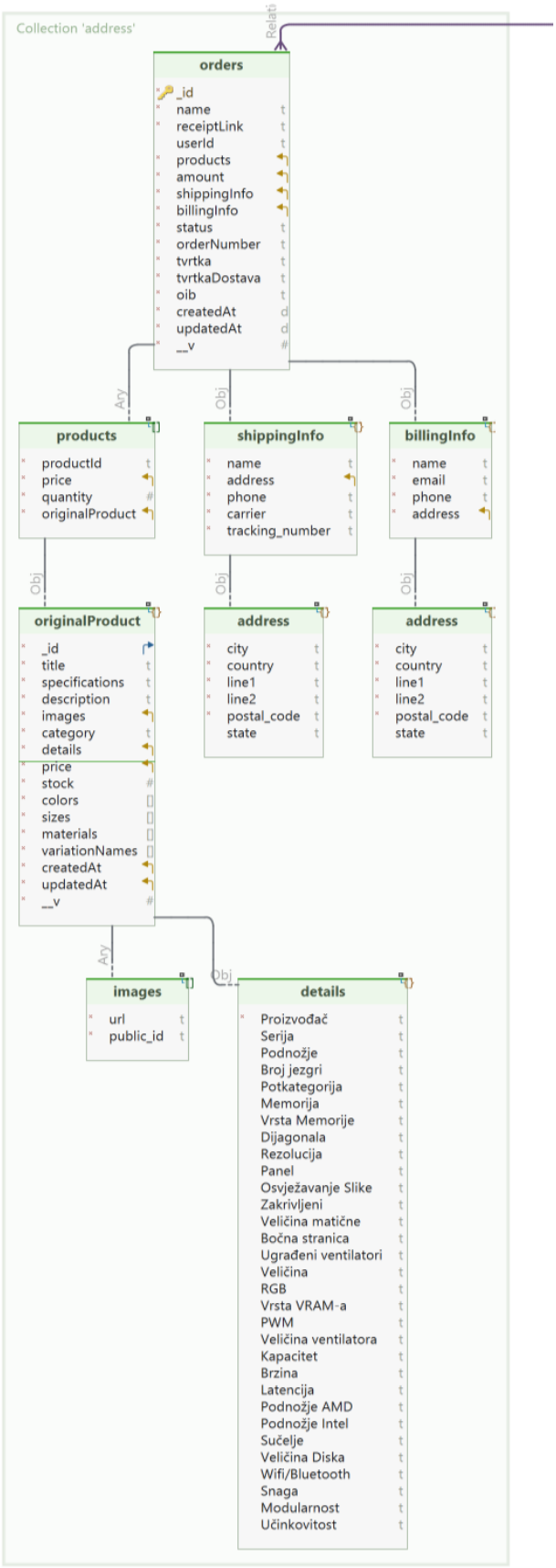
[16] „Introduction to Node.js“, https://nodejs.org/en/learn/getting-started/introduction-to-nodejs (posjećeno 25.8.2024)

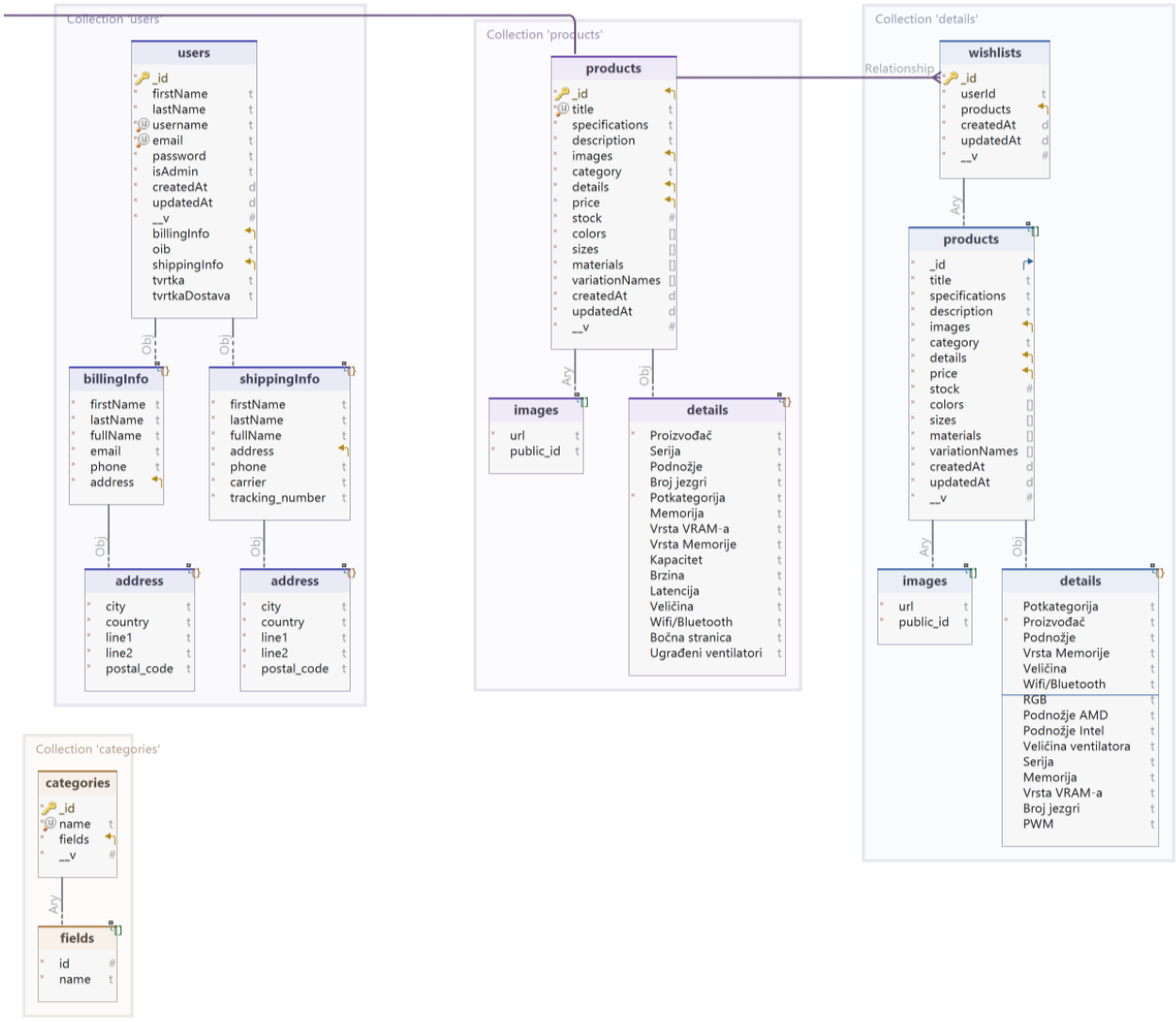
[17] „The Node.js Event Loop“, https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick (posjećeno 25.8.2024)

[18] „Context Switching in Operating System“, https://www.geeksforgeeks.org/context-switch-in-operating-system/ (posjećeno 25.8.2024)

[19] „Getting Started with Express JS“, https://www.geeksforgeeks.org/getting-started-with-express-js/ (posjećeno 25.8.2024)

# Dodaci


Collection 'address'
orders
_id
name
receiptLink
userId
products
amount
shippingInfo
billingInfo
status
orderNumber
tvrtka
tvrtkaDostava
oib
createdAt
updatedAt
__v
Any
Obj
Obj
products
productId
price
quantity
originalProduct
shippingInfo
name
address
phone
carrier
tracking_number
billingInfo
name
email
phone
address
Obj
Obj
Obj
originalProduct
_id
title
specifications
description
images
category
details
price
stock
colors
sizes
materials
variationNames
createdAt
updatedAt
__v
address
city
country
line1
line2
postal_code
state
address
city
country
line1
line2
postal_code
state
Ary
Obj
images
url
public_id
details
Proizvođač
Serija
Podnožje
Broj jezgri
Potkategorija
Memorija
Vrsta Memorije
Dijagonala
Rezolucija
Panel
Osvježavanje Slike
Zakrivljeni
Veličina matične
Bočna stranica
Ugrađeni ventilatori
Veličina
RGB
Vrsta VRAM-a
PWM
Veličina ventilatora
Kapacitet
Brzina
Latencija
Podnožje AMD
Podnožje Intel
Sučelje
Veličina Diska
Wifi/Bluetooth
Snaga
Modularnost
Učinkovitost




**Dodatak 1:** Model baze podataka

```
openCart: (state) => {
     state.open = true;
},
closeCart: (state) => {
     state.open = false;
},
resetCart: (state) => {
     state.open = false;
     (state.products = []), (state.quantity = 0), (state.totalPrice = 0);
},
removeProduct: (state, action) => {
     state.quantity -= action.payload.quantity;
     for (let i = 0; i < state.products.length; i++) {
          if (
               state.products[i]._id === action.payload.id &&
               state.products[i].color === action.payload.color
          ) {
               state.products.splice(i, 1);
          }
```

```
    }
    let priceToRemove = action.payload.price * action.payload.quantity;
    let tempCartTotal =
        Number.parseFloat(state.totalPrice) -
Number.parseFloat(priceToRemove);
    state.totalPrice = tempCartTotal;
},
incrementProductQuantity: (state, action) => {
    state.products = state.products.map((product) => {
        if (product._id === action.payload.id) {
            product.quantity += 1;
            state.quantity += 1;
            let priceChange = product.price * product.quantity;
            let tempCartTotal =
                Number.parseFloat(state.totalPrice) +
                Number.parseFloat(priceChange);
            state.totalPrice = tempCartTotal;
        }
        return product;
    });
},
decrementProductQuantity: (state, action) => {
    state.products = state.products.map((product) => {
        if (product._id === action.payload.id) {
            product.quantity = product.quantity > 1 ? product.quantity
- 1 : 1;
            state.quantity = state.quantity > 1 ? state.quantity - 1 :
1;
            let tempCartTotal =
                state.totalPrice > product.price
                    ? state.totalPrice - product.price
                    : product.price;
            state.totalPrice = tempCartTotal;
        }
        return product;
    });
},
```

**Dodatak 2:** Izvorni kôd za upravljanje proizvodima u košarici

```
exports.stripeWebHook = async (req, res) => {
    let signSecret = process.env.WEBHOOK_SECRET;
    const payload = req.body.toString();
    const sig = req.headers['stripe-signature'];
    let event;
    try {
        event = stripe.webhooks.constructEvent(payload, sig,
signSecret);
    } catch (error) {
        console.log(error.message);
        return res.status(400).send({ success: false, error: error });
    }
    switch (event.type) {
        case 'charge.succeeded':
            const charge = event.data.object;
            const paymentIntent = await stripe.paymentIntents.retrieve(
                charge.payment_intent
            );
```

```
                const customerEmail = charge.billing_details.email;
                const invoice = await stripe.invoices.retrieve(
                     paymentIntent.metadata.invoice_id
                );
                const invoiceItems = invoice.lines.data;
                const name = charge.billing_details.name;
                const amount = charge.amount / 100;
                const billingInfo = charge.billing_details;
                const shippingInfo = charge.shipping;
                const recieptUrl = charge.receipt_url;
                const products = [];
                for (var i = invoiceItems.length - 1; i >= 0; i--) {
                     const productFromDb = await Product.findById(
                          invoiceItems[i].metadata.productId
                     );
                     const product = {
                          productId: invoiceItems[i].metadata.productId,
                          color: invoiceItems[i].metadata.color,
                          price: invoiceItems[i].metadata.price,
                          quantity: invoiceItems[i].amount,
                          originalProduct: productFromDb,
                     };
                     products.push(product);
                     await Product.findByIdAndUpdate(
                          { _id: invoiceItems[i].metadata.productId },
                          { $inc: { stock: -invoiceItems[i].amount } }
                     );
                }
                var orderNumber = Math.floor(Math.random() * 9000000000) +
1000000000;
                const order = new Order({
                     name: name,
                     receiptLink: recieptUrl,
                     products: products,
                     amount: amount,
                     shippingInfo: shippingInfo,
                     billingInfo: billingInfo,
                     status: 'Paid',
                     orderNumber: orderNumber,
                     tvrtka: paymentIntent.metadata.tvrtka,
                     tvrtkaDostava: paymentIntent.metadata.tvrtkaDostava,
                     oib: paymentIntent.metadata.oib,
                     userId: paymentIntent.metadata.userId,
                });
                await order.save();
                const mailOptions = {
                     from: 'email@email.com',
                     to: customerEmail,
                     subject: 'Switchy - Order receipt',
                     html: generateReceipt(
                          recieptUrl,
                          order._id,
                          orderNumber,
                          amount,
                          products
                     ),
                };
                mailTransport().sendMail(mailOptions, function (err, info)
{
                     if (err) {
```

```
                        console.log(err);
                    } else {
                        res.json({ success: true, message: 'Email sent!'
});
                    }
                });
                console.log('Payment was successful!');
                break;
    }
    return res.status(200).send({ success: true });
};
```

**Dodatak 3:** Izvorni kôd za integraciju paketa Stripe